AOSP 进阶丛书 · 第八册 · ANDROID 13 (API 33)

图形渲染深潜流水线全解

BufferSlot 四态状态机、BLAST 每帧原子事务、SurfaceFlinger 的 commit/composite 内景、HWC 直通决策、Fence 依赖图与帧率投票——一个像素从 App 到面板的完整白盒。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目录

前言：从"会用图"到"管流水线" 2

第一章 全链路总览：一个像素的旅程 2

第二章 图形内存：Gralloc 与 GraphicBuffer 3

第三章 BufferQueue：缓冲状态机 4

第四章 BLAST：把帧升级为原子事务 7

第五章 Surface 与 Transaction：App 侧的合同 9

第六章 SurfaceFlinger 内景：commit 与 composite 10

第七章 合成决策：Overlay 直通 vs GPU 11

第八章 Fence：跨设备时序的艺术 13

第九章 设计决策九问 13

附录 A 观测实操 14

附录 B 快问快答（18 条） 14

附录 C 自检题 15

附录 D 动手实验室（一天量） 15

第十章 VSync 调度深潜：从硬件脉冲到 commit 的完整唤醒链 16

第十一章 Gralloc 分配链：一块 GraphicBuffer 的诞生 19

第十二章 RenderEngine：CLIENT 层合成的 GPU 管线 22

第十三章 帧的生命周期闭环：一块 buffer 的六站之旅 26

教科书习题集（分级） 30

术语表 33

排障手册：图形渲染深潜问题定位 35

附录 E：图形渲染核心机制深度速查表 37

前言：从"会用图"到"管流水线"

第五册把画面交到了 SurfaceFlinger 门口就停了——本册推门进去：**缓冲如何轮转（BufferQueue 状态机）、帧如何原子提交（BLAST 事务）、SF 如何合成一帧（commit/composite）、硬件如何直通（HWC 决策）、跨设备的时序如何对齐（Fence）**。这五件事是图形岗面试与调优的全部主战场。

体例与前册一致：真实源码逐行注释 + 完整调用链 + 流程图。源码基线不变（本树 Android 13）；GUI=frameworks/native/libs/gui、SF=frameworks/native/services/surfaceflinger。读码路线：状态机（第三章）→ 事务（第四章）→ 合成（第六七章）→ fence（第八章），每章是上一章的下一段流水。

第一章 全链路总览：一个像素的旅程

设计洞察 · 学习目标

1. 说出图形流水线四段的所有者与两样跨段信物（缓冲句柄与 fence）；2. 画出 App→BLAST→SF→HWC 的单帧时序并标注双 VSync 位置；3. 解释"四段之间不传像素指针不等待"的原因；4. 给出读码路线图（3→4→6→7 章）



图 1 图形渲染全链路：App 生产→BLAST 提交→SF 合成→HWC 送显四段，含双 VSync 与三种旅行方式

四段流水各有一位"所有者"：App 拥有生产节拍（Choreographer，书5 §四），BLAST 拥有提交语义（事务），SF 拥有合成世界（Layer 树），HWC 拥有最终带宽（直通/叠加）。**四段之间不传像素指针不等待，只传两样东西：缓冲句柄与 fence**——这是整册的第一心法。

第二章 图形内存：Gralloc 与 GraphicBuffer

> **设计洞察 · 学习目标**
> 1. 解释 usage bits 如何决定内存落在哪个物理堆；2. 陈述跨进程零拷贝的真相（一份物理内存三个视图）；3. 避免 stride/格式泄漏类斜纹 bug；4. 追踪 GraphicBuffer 懒分配的触发时机（深化见第十一章）

一切从一块内存开始：`GraphicBuffer`（GUI/GraphicBuffer.cpp）经 **Gralloc**（hardware/interfaces/graphics/{allocator,mapper}，AIDL 化）分配——allocator HAL 出 dma-buf 句柄，mapper HAL 负责跨进程映射（`lockAsync`）。三个读码要点：

- **usage bits 决定内存去向**（GPU 纹理/扫描输出/编码器输入各走各的堆）——视频层能 Overlay 的物理前提是 buffer 落在控制器可见的堆；
- **跨进程零拷贝的真相**：同一 dma-buf 被 App（画）、SF（合成）、控制器（扫描）各自映射——**一份物理内存三个视图**；
- stride（行距）与格式（RGBA8888/P010…）： mapper 层概念泄漏到用户的常见坑（读图像出现斜纹= stride 算错）。

> **源码锚点 · 第二章锚点**
> GUI/GraphicBuffer.cpp：allocate/lockAsync 封装
hardware/interfaces/graphics/allocator/aidl：分配 HAL
hardware/interfaces/graphics/mapper/aidl：映射 HAL
hardware/interfaces/graphics/bufferqueue/2.0：13 新 AIDL BQ 试点（媒体域直连）

2.1 GraphicBuffer 的一生（逐段注释）

cpp GUI/GraphicBuffer.cpp（骨架+讲解注释）

```
// 讲解注释①：构造仅记尺寸/格式/usage——懒分配：
// 真正的分配在首次 dequeue 需要"新槽"时（BufferQueueCore 请求）。
// GraphicBuffer::allocate():
// // 讲解注释②：走 Gralloc allocator（AIDL）：
// // "我要 width×height 的 P010、usage 含 GPU_SAMPLED+COMPOSER_OVERLAY"
// // HAL 返回 bufferId + handle（dma-buf 句柄）——内存在这里诞生。
// GraphicBuffer::lockAsync(..., outFence):
// // 讲解注释③：mapper HAL 按当前进程映射 + 返回可写地址（带 fence 依赖）；
// // 跨进程方（SF/控制器）各自 lockAsync 同一 bufferId——同一份物理页。
// GraphicBuffer::unlockAsync():
// // 讲解注释④：产生新的 acquire fence（GPU/驱动的写完承诺）。
```

先修要求与学习路线

先修：已读第一册（架构总览）。

读完本书你将能够：

- 解释图形渲染深潜的核心机制与设计决策
- 追踪关键调用链并标注每次跨进程跳
- 使用 dumpsys/日志定位本领域问题
- 回答书7 对应章节的面试题

> **设计洞察 · 逐段精读：懒分配的用意**
> 分配是昂贵且可能失败的操作（ION/dmabuf 池有限）；把分配推迟到"确实需要第 N 个缓冲"，池就能随帧率需求自适应——慢速 UI 两块就够，视频三块，录屏再加。`dumpsys SurfaceFlinger --dump-buffer` 里看到的槽位增长史就是这段逻辑的现场。

第三章 BufferQueue：缓冲状态机

> **设计洞察 · 学习目标**
> 1. 默画 FREE/DEQUEUED/QUEUED/ACQUIRED 四态及每步携带的 fence；2. 解释 dequeue 的两种等待语义（等分配/等 release）；3. 区分 mFreeSlots 与 mFreeBuffers 的复用含义；4. 用 maxDequeuedBufferCount 解释三缓冲的谈判本质

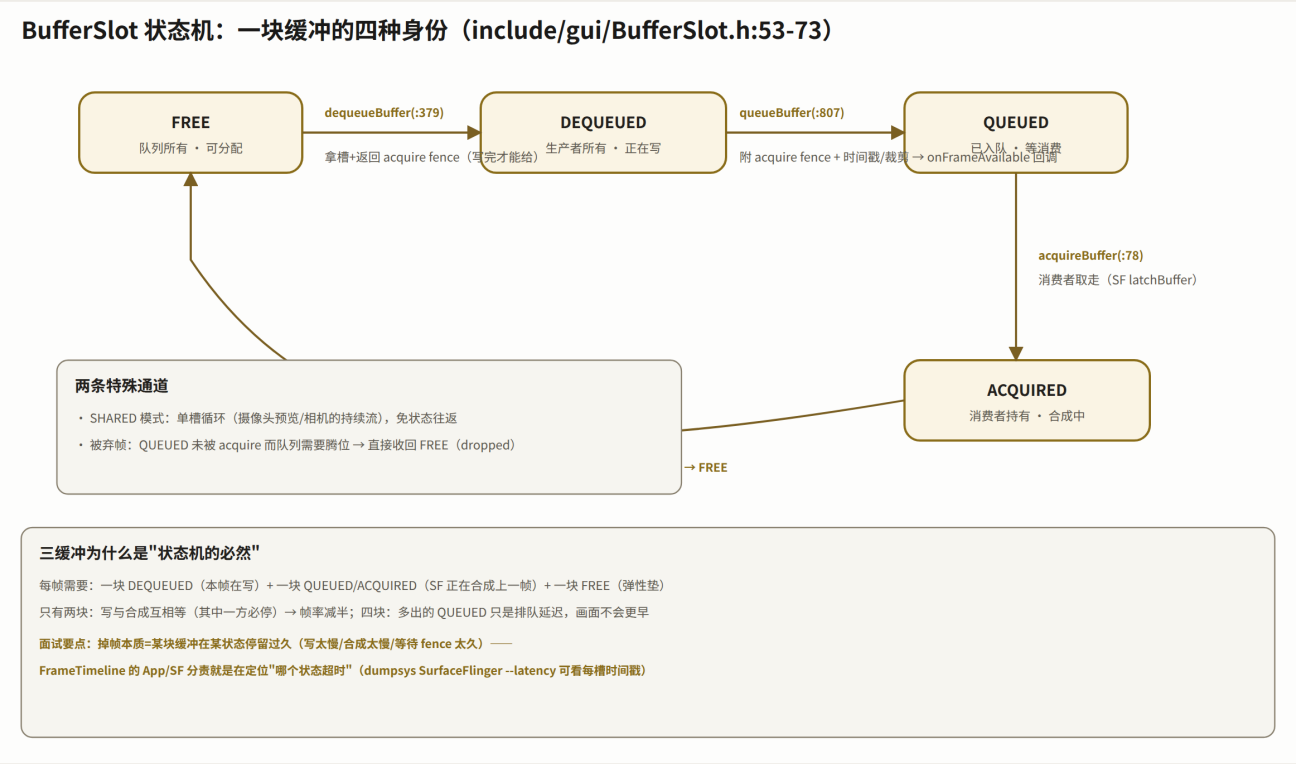


图 2 BufferSlot 状态机：FREE/DEQUEUED/QUEUED/ACQUIRED 四态流转与 SHARED 特殊通道，三缓冲的必然性

3.1 生产者两步逐行注释（真实源码）

cpp GUI/BufferQueueProducer.cpp:379-382 起（骨架+讲解注释）

```
status_t BufferQueueProducer::dequeueBuffer(int* outSlot,
sp<android::Fence>* outFence, uint32_t width, uint32_t height,
PixelFormat format, uint64_t usage, uint64_t* outBufferAge,
FrameEventHistoryDelta* outTimestamps) {
ATRACE_CALL();
// 讲解注释①：与消费者（SF）同锁——槽位表是两端共享账本，
// 这把锁就是"生产消费互斥"的全部实现（无内核对象）。
{ std::lock_guard<std::mutex> lock(mCore->mMutex);
...
// 讲解注释②：找 FREE 槽（或可复用的同尺寸槽）；没有则按
// maxDequeuedBufferCount 决定阻塞或返回 WOULD_BLOCK——
// App 侧 dequeue "卡住"的机制层就是这里在等 release。
...
// 讲解注释③：返回 slot + outFence（该槽上一任读者的 release
// fence）：GPU 编队时挂上依赖——fence 未到就写=花屏。
...
```

cpp GUI/BufferQueueProducer.cpp:807 起（骨架+讲解注释）

```
status_t BufferQueueProducer::queueBuffer(int slot, ...) {
// 讲解注释④：槽位 DEQUEUED→QUEUED；登记 acquire fence、
// crop/scale/时戳——这一帧的"自我描述"全部入账。
// 讲解注释⑤：唤醒消费者：mCore->mDequeueCondition /
// onFrameAvailable 回调（BLAST 模型里就是 BBQ:635 的入口）。
// 讲解注释⑥：返回 FrameEventHistoryDelta——帧时间线回执
//  （书5 FrameTimeline 的 App 侧数据源）。
}
```

3.3 BufferQueueCore：账本与三条队列

text GUI/bufferqueue/BufferQueueCore（结构+讲解注释）

```
mSlots[64] // 槽位数组上限 64——物理账本
mFreeSlots // 空槽（从未分配或已重置）——可触发新分配
mFreeBuffers // 已分配且 FREE 的缓冲——复用免分配（懒分配的另一半）
mQueue // QUEUED 帧队列（FIFO； acquire 侧按序取）
mDequeueCondition // 背压等待点（waitForFreeSlotThenRelock 的床）
mAbandoned // 消费者死了（SF 侧 Surface 销毁）→ 生产端 dequeue 报 NO_INIT
// 讲解注释：mFreeSlots 与 mFreeBuffers 的区分就是"要不要分配"的区分——
// 复用已分配缓冲零成本，所以稳态下 dequeue 永远走 mFreeBuffers。
```

设计洞察 · 逐段精读：maxDequeuedBuffers 的谈判

生产者可请求提高在途上限（IGraphicBufferProducer::setMaxDequeuedBufferCount），但需消费者点头（SF 按内存预算裁决）——三缓冲不是写死的三，是**上限谈判后的稳态**。SurfaceView 常配更多在途（视频解码超前），聊天列表两块即稳。

设计洞察 · 逐段精读：一把互斥锁的两面

①的共享账本意味着：dequeue 与 SF 的 acquire/release 在**同一进程空间**（BQ 的两端只在 App↔SF 之间——BINDER 只用于建立与控制，数据面不走 binder）。跨进程的生产消费（媒体编解码→App 显示）是 13 新 AIDL BQ 试点要解的题（第二章锚点）。记住分层：**同进程用锁，跨进程用事务，跨设备用 fence**。

3.1.1 dequeueBuffer 找槽段逐行注释（真实源码）

cpp GUI/BufferQueueProducer.cpp:411-446（节选+讲解注释）

```
{ // Autolock scope
std::unique_lock<std::mutex> lock(mCore->mMutex);
// 讲解注释⑤：无空闲槽且他人在分配中→等待（分配串行化，
// 防止并发分配同一池）——"dequeue 卡住"的第二种现场。
if (mCore->mFreeBuffers.empty() && mCore->mIsAllocating) {
mDequeueWaitingForAllocation = true;
mCore->waitWhileAllocatingLocked(lock);
...
}
// 讲解注释⑥：usage 合并消费者的要求（SF 侧声明"我要能直通的堆"）——
// 生产者未声明的能力由消费者补齐，分配才落在正确的物理堆。
usage |= mCore->mConsumerUsageBits;
// 讲解注释⑦：默认尺寸 + 旋转 90° 时宽高互换（transform hint 预转）。
...
int found = BufferItem::INVALID_BUFFER_SLOT;
while (found == BufferItem::INVALID_BUFFER_SLOT) {
// 讲解注释⑧：核心找槽循环：FREE 优先；不足则按
// maxDequeuedBufferCount 决定阻塞等 release 或丢弃最老 QUEUED。
status_t status = waitForFreeSlotThenRelock(FreeSlotCaller::Dequeue, lock, &found);
...
```

逐段精读⑤与⑧的两种等待语义：等**分配**（罕见、一次性）vs 等 **release**（常态、背压机制）。当 App 生产过快（游戏高帧率），⑧就是把生产速度压回消费速度的节流阀——**背压不在协议里，在状态机里**。

3.2 消费者两步

`BufferQueueConsumer::acquireBuffer`（:78）：按序取 QUEUED 帧（acquire fence 一并交出，SF latch 后在 composite 时真正等待）；`releaseBuffer`：挂 release fence 归还 FREE——**归还≠立即可写**，要等消费者读完（fence 才是最终话语权）。SHARED 模式（BufferSlot.h 注释末段）：单槽循环供相机持续流，免状态机往返。

第四章 BLAST：把帧升级为原子事务

设计洞察 · 学习目标

1. 陈述旧 BQ 双通道（内容/几何）的三类错配；2. 逐段讲清 onFrameAvailable 的影子队列与丢帧保节拍；3. 解释三段回执（latched/presented/released）各自的触发者；4. 用 desiredPresentTime 描述帧 pacing 路径



图 3 BLAST 模型：旧 BQ 直连的三问题与每帧一事务的新世界，帧责任链三段回执

4.1 onFrameAvailable 逐段注释（真实源码骨架）

cpp GUI/BLASTBufferQueue.cpp:635 起（骨架+讲解注释）

```
void BLASTBufferQueue::onFrameAvailable(const BufferItem& item) {
// 讲解注释①：每帧独立 Transaction（循环取数复用，池化）；
// BBQ 自持一个"阴影 SurfaceControl"代表本窗口内容。
// 讲解注释②：t->setBuffer(...) 挂缓冲+acquire fence；
// setTransactionCompleteCallback → 帧三段回执
// （latched/presented/最终 released）——FrameTimeline 原料。
// 讲解注释③：t->setDesiredPresentTime：未来帧可预约
// （游戏帧 pacing 的官方路径）；
// 讲解注释④：t->apply() 提交（SurfaceComposerClient:958）
// → SF 事务队列；本函数返回≠帧生效，等回执。
}
```

update（:205）则处理窗口几何变化（尺寸/变换）——**几何与内容终于同车**。旧模型（≤11）内容走 BQ、几何走 WMS 两条道导致"新内容旧位置"闪烁；BLAST 把帧定义为原子单元，是 12 以来动画/多窗正确性的地基。

4.1.1 onFrameAvailable 真实逐段注释（源码 :635-690）

cpp GUI/BLASTBufferQueue.cpp:635 起（节选+讲解注释）

```
void BLASTBufferQueue::onFrameAvailable(const BufferItem& item) {
{
std::unique_lock _lock{mMutex};
const bool syncTransactionSet = mTransactionReadyCallback != nullptr;
// 讲解注释①：同步事务模式（WMS 与 BBQ 的同步握手）：
// 若上一同步事务里还占着 buffer，先 getAndClear+releaseBuffer
// 腾出槽——避免新帧无槽可用。
if (syncTransactionSet) {
if (!mAcquireSingleBuffer) {
auto bufferData = mSyncTransaction->getAndClearBuffer(mSurfaceControl);
if (bufferData) { releaseBuffer(...); }
}
}
// 讲解注释②：影子队列记账：mNumFrameAvailable++；
// 若在等回调且积压≥2，主动 acquireAndReleaseBuffer 丢弃旧帧
// ——BLAST 也有"丢帧保节拍"的快速通道（对应状态机的 dropped）。
mNumFrameAvailable++;
if (mWaitForTransactionCallback && mNumFrameAvailable >= 2) {
acquireAndReleaseBuffer();
}
...
```

设计洞察 · 逐段精读：影子队列是什么

BLAST 在 App 进程里自建一层小队列（shadow queue）：帧先入此队，再按节拍逐帧组 Transaction 提交 SF——**提交速率与生产速率解耦**。这解释了两个现象：动画卡时 App 仍流畅生产（影子队垫着）；连续两帧积压时旧帧被主动丢弃（宁可跳帧不排队延迟）。

第五章 Surface 与 Transaction：App 侧的合同

设计洞察 · 学习目标

1. 区分 Surface（画布代言）与 SurfaceControl（属性遥控器）；2. 陈述 Transaction::apply 的三重语义（原子/异步/可回执）；3. 解释合并单位是"同一 vsync 周期"的机制保证；4. 说明动画与内容如何同帧生效

App 拿到的 `Surface`（GUI/Surface.cpp）是"带队列的画布代言"：`lockCanvas`（软件）/经 `Surface` 的 GPU 目标（硬件）最终都落到 dequeue→queue。**SurfaceControl 才是"窗口属性遥控器"**（书5 已见 WMS 侧），而 `SurfaceComposerClient::Transaction::apply`（:958）是两世界之间的签收台：buffer、几何、动画参数（书5 §8 的 RenderThread 动画就在这里生效）打包成一次原子提交。

text apply 的三重语义（读码后总结）

```
原子性：多参数一次生效（无中间态可见）
异步性：入 SF 队列即返回，生效在下个 commit
可回执：TransactionComplete回调链（BLAST 三段）供帧归因
```

5.1 事务的批处理与合并

text Transaction 的三种用法（读码归纳）

```
单帧提交（BLAST）：buffer+几何+回执一车——每帧一次
纯属性提交（动画）：只改矩阵/alpha（RenderThread 动画）——可多帧高频
批合并：Transaction::merge / SF 侧事务队列按 vsyncId 分桶，
同帧内多个来源（WMS 几何+App 内容+动画）在 commit 时一次性结算
// 讲解注释：合并的单位是"同一 vsync 周期"——迟到者进下一桶，
// 这就是 apply 返回≠生效、且动画与内容永远同帧生效的机制保证。
```

6.3 多屏与 VirtualDisplay

每块屏一个 DisplayDevice + 独立 composite（注释十二：帧率策略 per-display）；**VirtualDisplay**（投屏/录屏/副屏模拟）是"没有面板的屏"：composite 输出到自己的 BufferQueue（由请求方消费），节拍自定（录屏常 30fps 桶）。排障提示：投屏卡顿别查主屏 vsync——查 VirtualDisplay 的消费者（投屏协议端）是否及时 release。**镜像**（secure display）则是主屏 composite 的旁路复制，成本最低但布局同主屏。

源码锚点 · 第五章锚点

GUI/SurfaceComposerClient.cpp:958 Transaction::apply
GUI/Surface.cpp：生产者封装（dequeue/queue 的 App 门面）
GUI/BLASTBufferQueue.cpp:205/635：update/onFrameAvailable

第六章 SurfaceFlinger 内景：commit 与 composite

设计洞察 · 学习目标

1. 复述 commit→composite 的一帧主干与各自输入输出；2. 解释 latch 取旧帧的取舍理由（帧节拍优先）；3. 说明 EventThread 点播台模型与按需唤醒；4. 指出 VirtualDisplay 走 offscreen 结算的独立节拍（深化见第十/十三章）



图 4 SF 一帧内景：唤醒取件→commit（事务+latch+脏区）→composite（合成计划→HWC→present），Layer 两面与离屏层

`composite`（SF/SurfaceFlinger.cpp:2164）每 SF VSync 执行；`commitTransactionsLocked`（:3119）结算本帧世界：应用 Layer 几何、**latchBuffer**（取每层最新 QUEUED 缓冲，acquire fence 未到就用旧帧——帧率优先于新鲜度）、计算脏区。`commitOffscreenLayers`（:3512）单独结算不挂屏的层（录屏/虚拟屏走自己的节拍）。

6.1 VSync 分发逐段注释（Scheduler/EventThread.cpp:434 起）

cpp SF/Scheduler/EventThread.cpp:434-470（节选+讲解注释）

```
void EventThread::threadMain(std::unique_lock<std::mutex>& lock) {
while (mState != State::Quit) {
// 讲解注释①：事件来源三种：HOTPLUG（屏插拔，重置 vsync 态）、
// VSYNC（节拍本体）、MODE_CHANGE。逐个从 pending 队列出。
if (!mPendingEvents.empty()) {
event = mPendingEvents.front(); mPendingEvents.pop_front();
```

```
switch (event->header.type) {
case DISPLAY_EVENT_HOTPLUG: ... // 讲解注释②：热插拔时
case DISPLAY_EVENT_VSYNC: ... // 建立/重置 mVSyncState
}
}
// 讲解注释③：遍历连接（connection）按各自 vsyncRequest 派发：
// App 侧的 DisplayEventReceiver 即某条 connection 的 BitTube
// 对端（:179 stealReceiveChannel 交给 Choreographer 世界）。
// 请求分级：None/Single/Periodic——按需唤醒，没人要就不发。
...
```

逐段精读③的"按需分发"：EventThread 不是广播塔而是**点播台**——App 只有注册了 vsync 回调（Choreographer postCallback 之后）才会收到，SF 自己的调度也走同类机制（另一 EventThread 实例）。这是"没人监听的节拍不产生开销"的节能设计，也解释了 idle 时 perfetto 里 vsync 轨稀疏的原因。

6.2 composite 骨架逐段（:2164 起）

cpp SF/SurfaceFlinger.cpp:2164（骨架+讲解注释）

```
void SurfaceFlinger::composite(nsecs_t frameTime, int64_t vsyncId) {
// 讲解注释①：时间戳与 vsyncId 贯穿全帧——FrameTimeline 归因的
// 关联键（一次 composite 对应一组 presented 帧）。
// 讲解注释②：CompositionEngine::present(CompositionRefreshArgs)：
// 把"本帧 Layer 快照 + 输出屏集合"交给策略引擎（CE 层），
// SF 自身只管世界状态，不直接碰 HWC——职责分离的合成器版。
// 讲解注释③：结束后 sampleFrames? （LayerHistory 采样本帧各层
// 提交间隔）→ 帧率策略更新（自适应刷新的输入）。
// 讲解注释④：present fence 回收 → 通知各 Layer 的 releaseCallback
// → BLAST 三段回执的 presented 段在此触发。
}
```

设计洞察 · 逐段精读：为什么 latch 不等 fence

GPU 写完的 acquire fence 可能还要几十微秒；若 latch 阻塞等待，SF 主循环卡住，**整帧**延迟——代价是本层这帧用旧内容（stale frame，肉眼几乎无感）。这是"内容新鲜度 vs 帧节拍稳定"的经典取舍，也是 fence 设计成"事后声明"而非"事前同步"的原因（第八章呼应）。

第七章 合成决策：Overlay 直通 vs GPU

设计洞察 · 学习目标

1. 走一遍候选→validate→CLIENT 兜底的三步决策；2. 列出决策变量（混合/缩放/格式/层数上限）并解释为何必须逐帧决策；3. 用省电三理由说明直通的价值；4. 描述 LayerHistory 投票的单位与迟滞（GPU 兜底的实现见第十二章）

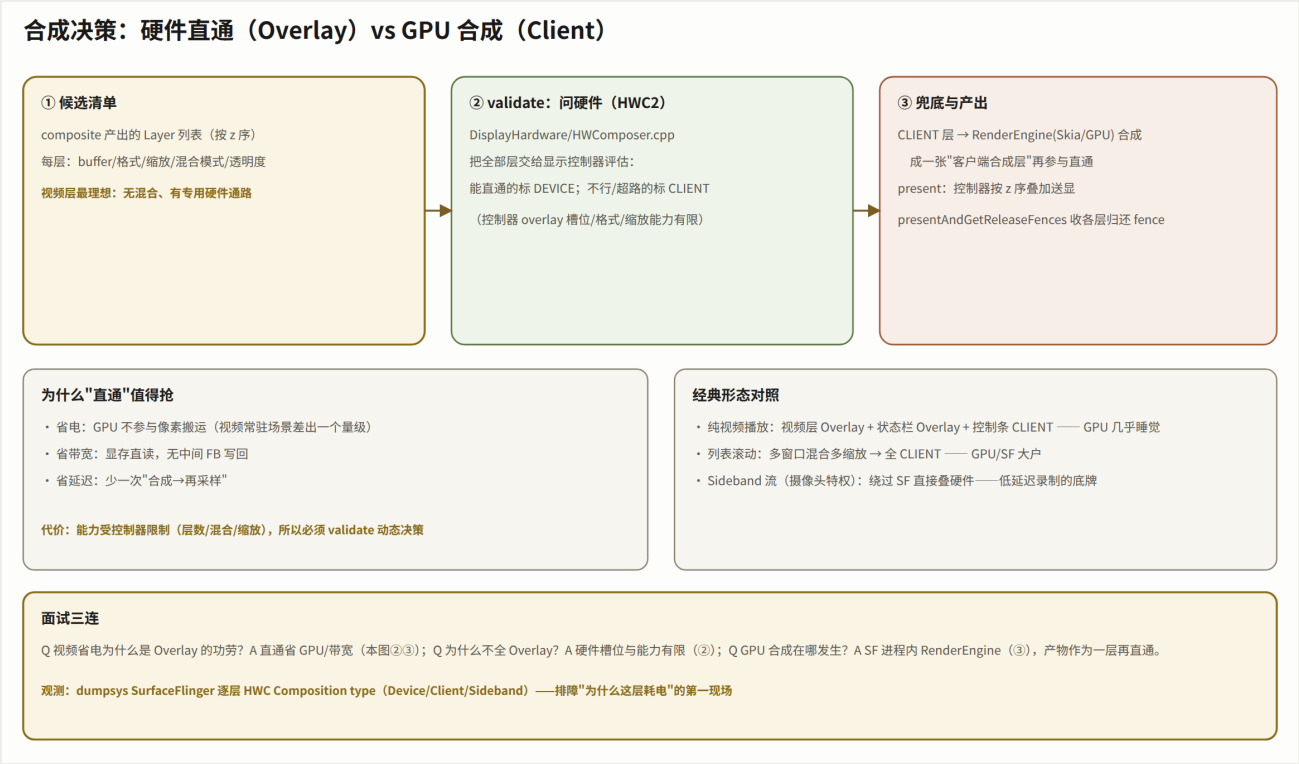


图 5 HWC 合成决策：候选→validate 硬件点名→CLIENT 兜底三步，省电三理由与经典形态对照

三步语义：composite 产出 Layer 清单 → `HWComposer`（SF/DisplayHardware/）validate 让控制器挑可直通层（标 DEVICE）→ 余层（CLIENT）经 **RenderEngine**（Skia GPU）合成一张再参与直通 → present 送显并收 present fence。决策变量：混合/缩放/格式/层数上限——**所以必须逐帧动态决策**而非静态分配。Sideband 流是特权旁路（相机低延迟录制）。

7.1 帧率策略：LayerHistory 投票（13 主线特性）

`Scheduler/LayerHistory.cpp` 持续采样每层"两次提交的间隔"（composite 注释③的采样点），按启发式归类（活跃层/空闲层）→ 结合 `FrameRateOverrideMappings`（配置的覆盖表）算出全局刷新率提案 → 经 DisplayModeController 切换面板模式。读码三要点：**投票单位是 Layer 不是进程**（一个视频层可把整机拉到 120Hz）；**切换有迟滞**（防抖动，进入高刷要连续 N 个快速帧）；**触摸事件即时提速**（InputFlinger 与 Scheduler 的联动点）——滑动跟手性的最后一环在这里。

7.2 RenderEngine：SF 进程内的 GPU 画师

CLIENT 层合成走 `SF/renderengine/skia/`（13 主实现）：Skia 录制各层 draw（纹理采样/混合/圆角裁剪/阴影）→ GPU 扌 行 → 输出帧缓冲层。读码两个要点：**缓存策略**（每层 ExternalTexture 复用 + GLCache 上下文池——合成是每帧全量重画，缓存的是资源不是结果）；**色彩管理**（dataspace 转换在 Skia 管线内完成——HDR/广色域内容在 CLIENT 路径才被正确 tone-map，这是 Overlay 与 CLIENT 画质差异的来源之一）。

第八章 Fence：跨设备时序的艺术

设计洞察 · 学习目标

1. 用内核 sync 文件语义解释 fence 的 fd 本质与 merge；2. 区分 acquire/release/present 三主角的方向与落点；3. 画出单帧三 fence 的时间轴；4. 应用排障口诀（花屏查 release、掉帧查延迟、"呈现了没"查 present）



图 6 Fence 三主角（acquire/release/present）、内核本质与一帧的 fence 时间轴

Fence（libs/ui/Fence.cpp:48 wait/:90 merge）是内核同步对象的 fd 封装，跨进程经 Binder fd 通道（书3 § 六）传递。核心是**不等待的依赖表达**：CPU 提交后即返回，GPU/控制器按依赖图在内核里排队唤醒。三主角：acquire（写完承诺）、release（读完承诺）、present（上屏事实）。排障口诀：**花屏查 release 依赖，掉帧查 fence 延迟轨，"呈现了没"查 present fence**。

第九章 设计决策九问

- **为什么缓冲轮转用状态机而非消息？** 槽位是有限资源，状态机天然表达"谁拥有谁负责"，且 FREE/DEQUEUED 归属即锁的边界（第三章）。
- **为什么三缓冲？** 状态机四个角色（写/排/读/垫）的最小完备配置——少则互等、多则纯延迟（第三章）。
- **为什么 BLAST 要事务化帧？** 消除内容/几何双通道错配；帧成为原子单元后回执、预约（desiredPresentTime）、批处理都有了支点（第四章）。

- **为什么 dequeue 返回 fence 而不是同步等？** GPU 流水线深度高，同步等=CPU 陪 GPU 干耗；依赖交给内核，CPU 继续下一帧准备（第三/八章）。
- **为什么 latch 用旧帧？** 帧节拍稳定性的优先级高于单层内容新鲜度（第六章）。
- **为什么 HWC validate 每帧都问？** 层组合能力硬件有限且动态变化（视频进出/透明度变化），静态分配必错（第七章）。
- **为什么 GPU 合成在 SF 进程？** 合成是与 Layer 世界观紧耦合的领域逻辑，进程内免跨进程缓冲搬运；HWC 才是跨进程/跨设备的边界（第七章）。
- **为什么 AIDL BQ 只先落媒体域？** 生产消费跨大进程高频直连只有媒体最痛；UI 域 BLAST 已够（第二/四章）。
- **为什么帧率策略放 SF（LayerHistory）？** 只有合成器看得到全部层的提交历史——投票权在信息最全处（第一章图注/第九章观察）。

附录 A 观测实操

目标	命令/工具	看点
Layer 全家福	dumpsys SurfaceFlinger	每层合成方式(DEVICE/CLIENT)/几何/缓冲状态
帧时戳	dumpsys SurfaceFlinger --latency	desiredPresent/present 时间戳序列
缓冲细节	dumpsys SurfaceFlinger --dump-buffer	各槽 fence/时戳（状态机现场）
掉帧归因	perfetto FrameTimeline	latched/acquire/presented 三段+责任标签
HWC 计划	dumpsys SurfaceFlinger --list-strategies / 逐层 type	直通命中情况
App 侧帧	Choreographer FrameCallback / FrameMetrics	书5 §四的对接点

附录 B 快问快答（18 条）

- BQ 数据面走 Binder 吗？——不走：同进程两端+锁；跨进程靠 dma-buf 映射与 AIDL 试点。
- dequeue 与 acquire 的 fence 方向？——dequeue 给 release（等上家读完）；acquire 给 acquire（等生产写完）。
- 三缓冲多分配在哪？——dequeue 分配 GraphicBuffer 按需增长（maxDequeue 限制）。
- BLAST 与旧 BQ 并存吗？——13 默认 BLAST；SurfaceView/媒体路径个别仍旧模型。
- desiredPresentTime 谁消费？——SF commit 侧按预约时间投放（游戏 pacing）。
- Transaction apply 是同步吗？——异步入队；同步需要 explicit 同步模式参数。

- latch 旧帧算丢帧吗？——该层 stale，但不整帧 jank；FrameTimeline 记该层。
- HWC CLIENT 合成产物去哪？——RenderEngine 画进 FB 层再直通。
- Sideband 与 Overlay 区别？——Sideband 绕 SF 合成直叠硬件（低延迟特权）。
- present fence 与 vsync 关系？——present=面板扫描输出事实；vsync=节拍源，两者不重合。
- GraphicBuffer 跨进程映射几次？——按需各进程一次（mapper）；物理内存一份。
- usage 不匹配的后果？——分配在错误堆/无法直通/性能差。
- AIDL BQ 2.0 解决什么？——媒体跨进程直连免中转（13 试点）。
- FrameTimeline 数据源？——BQ 帧事件历史+BLAST 回执+SF 合成记录三方合成。
- 录屏走哪个时钟？——VirtualDisplay 自队（offscreen 结算），非主屏 vsync。
- 帧率投票怎么算？——LayerHistory 各层提交率+heuristic，合成全局刷新率。
- 多屏合成一帧一次吗？——每 DisplayContent 独立 composite。
- 学完本册去哪？——书9 后台与电源（渲染功耗的另一半答案）。

附录 C 自检题

- 默画 BufferSlot 状态机与每步携带的 fence。
- BLAST 事务包含哪三类内容？三段回执分别由谁触发？
- composite 内 latch 与 HWC validate 的先后与理由。
- 一帧从 queueBuffer 到 present 的五步时间轴（含三种 fence 落点）。
- 为什么视频场景 GPU 几乎睡觉？用第七章决策三步说明。

附录 D 动手实验室（一天量）

实验一"状态机现场"：静止桌面时 `dumpsys SurfaceFlinger --dump-buffer` 记录槽位数与状态；播放视频再看（槽位增长/SHARED 出现）；滚动列表连抓三次（QUEUED/ACQUIRED 波动）——把第三章状态机逐槽对号。

实验二"直通侦查"：播放本地视频时 `dumpsys SurfaceFlinger | grep -A3 video_layer` 找 Composition type=DEVICE；给视频加个半透明遮罩层再看（混合迫使 CLIENT）——第七章决策三步的现场反转。

实验三"fence 时轴"：perfetto 抓 5s 滚动，打开 fence 与 FrameTimeline 轨：对每帧标出 acquire/release/present 三落点；找出一个 Jank(SF) 帧解释哪段超时。

实验四"BLAST 背压"：写 demo 连续 postCallback 高频提交（超出面板刷新率），观察影子队列丢弃（logcat BBQ 标签）与 dequeue 阻塞——第四章两段精读的实证。

实验五"帧率投票"：设置里切 60/120Hz，`dumpsys SurfaceFlinger --frame-rate-override` 对照 LayerHistory 的分层建议；放一个 24fps 视频观察策略（多数机型仍全局高刷、仅部分 LFD 屏做层帧率）。

第十章 VSync 调度深潜：从硬件脉冲到 commit 的完整唤醒链

设计洞察 · 学习目标

1. 画出"硬件 VSync→VSyncPredictor→VSyncDispatchTimerQueue→DispSyncSource→EventThread/MessageQueue"的完整分发链；2. 解释 VSyncRequest 四档（None/Single/SingleSuppressCallback/Periodic）的按需唤醒语义；3. 说明 SF 主循环为何不走 EventThread 而直接在 VSyncDispatch 上挂号；4. 用 perfetto 区分 VSYNC-app 与 VSYNC-sf 两条轨并归因唤醒延迟

第六章 6.1 已看过 EventThread::threadMain 的骨架，本章把它补全为一条完整唤醒链——从控制器硬件脉冲到 `SurfaceFlinger::commit` 被执行，中间每一跳都是独立的代码实体：

- **硬件入轨**：HWC 回调 `SurfaceFlinger::onVsyncReceived`（SF/SurfaceFlinger.cpp:1871）→ `mScheduler->addResyncSample(timestamp, vsyncPeriod, ...)`（:1883）——硬件时戳只做一件事：喂给**预测器校准模型**；
- **调度三件套**（VsyncSchedule.cpp:79/90/102）：Tracker（`VSyncPredictor`，滑动窗口回归预测未来 vsync 时刻）、Dispatch（`VSyncDispatchTimerQueue`，按预测时刻设 alarm 唤醒订阅者）、Controller（管理硬件开关与 resync）；
- **两类订阅者**：`DispSyncSource`（每条 EventThread 一个，CallbackRepeater 周期续约）与 `MessageQueue`（SF 主循环直连，注册名 `"sf"`，MessageQueue.cpp:118-127）；
- **关键架构事实**：Android 13 里 SF 自己的合成节拍**不经 EventThread**——`mScheduler->initVsync(...)`（SurfaceFlinger.cpp:3428）直接在 VSyncDispatch 上挂回调。EventThread 只服务两条线：`"app"`（:3417，Choreographer 的节拍源）与 `"appSf"`（:3422，SF 侧事件监听，带拦截器）。**点播台模式贯穿始终：谁订阅谁被唤醒，没人要就 Idle。**

10.1 threadMain 全文精读：事件循环与两帧唤醒技巧

cpp SF/Scheduler/EventThread.cpp:434-528（节选+讲解注释）

```
void EventThread::threadMain(std::unique_lock<std::mutex>& lock) {
DisplayEventConsumers consumers;
while (mState != State::Quit) {
std::optional<DisplayEventReceiver::Event> event;
// 讲解注释①：出队一个 pending 事件；VSYNC 类型若被拦截器
// 注册（"appSf" 线）则先喂 mInterceptVSyncsCallback 存 trace。
if (!mPendingEvents.empty()) {
event = mPendingEvents.front(); mPendingEvents.pop_front();
switch (event->header.type) {
case DISPLAY_EVENT_HOTPLUG: // 讲解注释②：热插拔维护
```

```
if (event->hotplug.connected && !mVSyncState)
mVSyncState.emplace(event->header.displayId);
else if (!event->hotplug.connected && mVSyncState &&
mVSyncState->displayId == event->header.displayId)
mVSyncState.reset(); // ——拔屏则清 vsync 态。
break;
case DISPLAY_EVENT_VSYNC:
if (mInterceptVSyncsCallback)
mInterceptVSyncsCallback(event->header.timestamp);
break;
}
}
// 讲解注释③：遍历连接表（弱引用，死了即擦除）：
// vsyncRequested 记录"还有没有人要节拍"——全部为 None
// 就允许线程进 Idle，源被关掉（见⑥）。
bool vsyncRequested = false;
auto it = mDisplayEventConnections.begin();
while (it != mDisplayEventConnections.end()) {
if (const auto connection = it->promote()) {
vsyncRequested |= connection->vsyncRequest != VSyncRequest::None;
if (event && shouldConsumeEvent(*event, connection))
consumers.push_back(connection);
++it;
} else { it = mDisplayEventConnections.erase(it); }
}
// 讲解注释④：命中的连接集中派发——postEvent 写 BitTube，
// 对端（App 的 DisplayEventReceiver / Looper fd）被唤醒。
if (!consumers.empty()) { dispatchEvent(*event, consumers); consumers.clear(); }
// 讲解注释⑤：状态机：有源有订阅→VSync；只有源→Idle；
// 屏幕断开但有人订阅→SyntheticVSync（软件伪造节拍）。
State nextState;
if (mVSyncState && vsyncRequested)
nextState = mVSyncState->synthetic ? State::SyntheticVSync : State::VSync;
else nextState = State::Idle;
// 讲解注释⑥：状态切换才动源开关——setVSyncEnabled 的
// 开/关最终落到 VsyncSchedule::Controller（硬件使能）。
if (mState != nextState) {
if (mState == State::VSync) mVSyncSource->setVSyncEnabled(false);
else if (nextState == State::VSync) mVSyncSource->setVSyncEnabled(true);
mState = nextState;
}
if (event) continue;
// 讲解注释⑦：无事件则睡条件变量；SyntheticVSync 态 16ms
// 超时自造 VSYNC（屏灭仍按 60Hz 喂 App），其余 1000ms
// ——超时即"驱动停跳"兜底：打 "Faking VSYNC due to
// driver stall" 并补一条合成事件，防止 App 永眠。
if (mState == State::Idle) { mCondition.wait(lock); }
else {
const std::chrono::nanoseconds timeout =
mState == State::SyntheticVSync ? 16ms : 1000ms;
if (mCondition.wait_for(lock, timeout) == std::cv_status::timeout) {
if (mState == State::VSync) {
ALOGW("Faking VSYNC due to driver stall for thread %s", mThreadName);
}
mPendingEvents.push_back(makeVSync(...));
```

```
}
}
}
}
```

"按需分发"的具体规则在 `shouldConsumeEvent`（EventThread.cpp:542-575）：`VSyncRequest::Single` 命中一帧后**自动降级为 SingleSuppressCallback**，下一帧消费一次但不派发、再降为 None——单次请求实际唤醒两轮。注释写明原因："Single wakes up for the next two frames to avoid scheduler overhead"：省掉"请求→响应"一次跨线程往返，代价是多醒一轮空转。

请求档位	语义	典型调用方
None	不订阅，Idle	后台无动画的 App
Single	醒来两帧（第二帧静默吞掉）	Choreographer 的 doFrame（每次 scheduleVsync）
SingleSuppressCallback	只醒一帧且不派发	Single 的自动降级态
Periodic	按 N 个周期派发一次（N=1 每帧）	长动画/视频帧回调

10.2 上游：CallbackRepeater 的周期续约与 DispSyncSource

cpp SF/Scheduler/DispSyncSource.cpp:86-105（节选+讲解注释）

```
void CallbackRepeater::callback(nsecs_t vsyncTime, nsecs_t wakeupTime, nsecs_t readyTime) {
{ std::lock_guard lock(mMutex);
mLastCallTime = std::chrono::nanoseconds(vsyncTime); } // 讲解注释①
mCallback(vsyncTime, wakeupTime, readyTime); // 讲解注释②：通知
// DispSyncSource→EventThread::onVSyncEvent（EventThread.cpp:393），
// 打包成 DISPLAY_EVENT_VSYNC 压入 mPendingEvents 并唤醒线程。
{ std::lock_guard lock(mMutex);
if (!mStarted) return; // 讲解注释③：期间被 stop 则不再续约
auto const scheduleResult = mRegistration.schedule(
{.workDuration = mWorkDuration.count(),
.readyDuration = mReadyDuration.count(),
.earliestVsync = vsyncTime}); // 讲解注释④：以本次
// vsync 为锚点预约下一次——wake 早于 vsync（workDuration
// 是留给 SF/App 的干活时长，readyDuration 是缓冲就绪余量）。
LOG_ALWAYS_FATAL_IF(!scheduleResult.has_value(), "Error rescheduling callback");
}
}
```

三个读码要点：①**两线不同时长**——`"app"` 线 workDuration 用 `configs.late.appWorkDuration`、`"appSf"` 用整帧周期（SurfaceFlinger.cpp:3417-3426），这正是 App 比 SF 早半帧唤醒的机制来源（第一章图的双 VSync）；②**自续约模型**——CallbackRepeater 每次回调自己 schedule 下一次，`setVSyncEnabled(false)` 只是把 mStarted 关掉，alarm 自然停摆；③**预测而非等待**——所有时刻来自 `VSyncPredictor` 的回归外推，硬件 vsync 只在 addResyncSample 时纠偏，误差大才强制 resync。

10.3 下游：MessageQueue 与 vsyncId 的诞生

cpp SF/Scheduler/MessageQueue.cpp:100-115 与 45-58（合并节选+讲解注释）

```
void MessageQueue::vsyncCallback(nsecs_t vsyncTime, nsecs_t targetWakeupTime,
nsecs_t readyTime) {
ATRACE_CALL(); // 讲解注释①：systrace 上的 "sf" 节拍轨即此处
{ std::lock_guard lock(mVsync.mutex);
mVsync.lastCallbackTime = std::chrono::nanoseconds(vsyncTime);
mVsync.scheduledFrameTime.reset(); }
// 讲解注释②：vsyncId 在这里诞生！TokenManager 按三个预测时刻
//  （唤醒/就绪/节拍）生成 int64 token——它是 FrameTimeline 全
// 帧归因的关联键（第六章 composite 注释①的源头在此）。
const auto vsyncId = mVsync.tokenManager->generateTokenForPredictions(
{targetWakeupTime, readyTime, vsyncTime});
mHandler->dispatchFrame(vsyncId, vsyncTime); // 讲解注释③
}
void MessageQueue::Handler::handleMessage(const Message&) {
mFramePending.store(false);
const nsecs_t frameTime = systemTime();
auto& compositor = mQueue.mCompositor;
// 讲解注释④：一帧的主干三连——commit 返回 false（上一帧
// present fence 未到、背压生效）则本帧直接放弃，等待
// commit 内部 scheduleCommit 重新预约，绝不硬扛。
if (!compositor.commit(frameTime, mVsyncId, mExpectedVsyncTime)) return;
compositor.composite(frameTime, mVsyncId);
compositor.sample(); // 讲解注释⑤：LayerHistory 采样点
}
```

设计洞察 · 设计洞察：为什么 SF 退出 EventThread 模型

EventThread 的价值在"一源多播、按需订阅"，但 SF 主循环永远只有一个消费者且每帧必醒——多播模型对它是纯开销。13 的做法：SF 直接持有 VSyncCallbackRegistration（MessageQueue.cpp:118 注册名 "sf"），workDuration 可被 VsyncModulator 按合成负载动态收放（早合成的 GPU 余量）；而 EventThread 降级为纯 App 侧与监听侧设施。perfetto 验证：VSYNC-sf 轨由 vsyncCallback 打点、VSYNC-app 轨由 "app" EventThread 打点——两条轨不同源、不同相，排查"SF 晚醒"时先看前者。

交叉阅读：6.1 的 threadMain 概览（本章给出全文与两帧唤醒技巧）、9 章第 7 问（为何 GPU 合成在 SF 进程——SF 节拍的自主性是前提）、第十三章的 commit/composite 实体即本链最末一跳。

第十一章 Gralloc 分配链：一块 GraphicBuffer 的诞生

设计洞察 · 学习目标

1. 复述"dequeue 触发分配→GraphicBufferAllocator→Gralloc4Allocator→AIDL/HIDL IAllocator→dma-buf→mapper import"的全链与每次跨进程跳；2. 解释 Android 13 allocator 走 AIDL 而 mapper 仍 HIDL 4.0 passthrough 的分工理由；3. 说明 descriptor 在 mapper 侧创建、分配在 allocator 侧执行的强制顺序；4. 用 dumpsys 与 strace 观测一次真实分配

第二章给了 GraphicBuffer 的鸟瞰与懒分配心法，本章走完它落到物理内存的每一步。先核实结论：**Android 13 的 allocator 已 AIDL 化**——接口定义在 hardware/interfaces/graphics/allocator/aidl（`android.hardware.graphics.allocator.IAllocator`，唯一方法 `AllocationResult allocate(in byte[] descriptor, in int count)`），**mapper 仍是 HIDL 4.0**（hardware/interfaces/graphics/mapper/ 下无 aidl 目录），且强制 passthrough 模式（Gralloc4.cpp:163 `LOG_ALWAYS_FATAL("gralloc-mapper must be in passthrough mode")`）。libui 的选型链在 GraphicBufferAllocator.cpp:51-63：Gralloc4 → Gralloc3 → Gralloc2 逐级探测。

链路位置：第三章 dequeueBuffer 找槽失败需要新缓冲时（`BUFFER_NEEDS_REALLOCATION`，BufferQueueProducer.cpp:548 `new GraphicBuffer(...)`）→ 构造内 `initSize` → 单例 `GraphicBufferAllocator::allocate`（libs/ui/GraphicBufferAllocator.cpp:176）→ `Gralloc4Allocator::allocate`（Gralloc4.cpp:1223）→ **跨进程跳 1**：AIDL binder 到 vendor 的 allocator HAL（或 HIDL 兜底）→ 返回 native_handle（dma-buf fd 数组）→ **跨进程跳 2**：`mMapper.importBuffer` 在本进程建立映射（mapper passthrough，同进程 so 调用）→ 之后 App 画、SF 合成、控制器扫描各自 lockAsync/import——同一物理页的多视图（第二章心法的实现层）。

11.1 Gralloc4Allocator 构造与 allocate 逐段精读（真实源码）

cpp libs/ui/Gralloc4.cpp:1194-1260（节选+讲解注释）

```
Gralloc4Allocator::Gralloc4Allocator(const Gralloc4Mapper& mapper) : mMapper(mapper) {
// 讲解注释①：HIDL 4.0 allocator 先拿——即使上 AIDL，descriptor
// 校验/导入仍靠 HIDL mapper，两个通道并行存在。
mAllocator = IAllocator::getService();
if (__builtin_available(android 31, *)) {
if (hasIAllocatorAidl()) { // 讲解注释②：AServiceManager_
// isDeclared(kAidlAllocatorServiceName) 静态缓存
//  （Gralloc4.cpp:102-110）——VINTF 声明即探测，免反复查询。
mAidlAllocator = AidlIAllocator::fromBinder(ndk::SpAIBinder(
AServiceManager_waitForService(kAidlAllocatorServiceName.c_str())));
ALOGE_IF(!mAidlAllocator, "AIDL IAllocator declared but failed to get service");
}
}
if (mAllocator == nullptr && mAidlAllocator == nullptr) {
ALOGW("allocator 4.x is not supported"); // 讲解注释③：双通道
return; // 全空→降级 Gralloc3
}
}
// ...
status_t Gralloc4Allocator::allocate(std::string requestorName, uint32_t width, ...
buffer_handle_t* outBufferHandles, bool importBuffers) const {
IMapper::BufferDescriptorInfo descriptorInfo;
if (auto error = sBufferDescriptorInfo(requestorName, width, height, format,
layerCount, usage, &descriptorInfo) != OK) {
return error; // 讲解注释④：请求者名（如窗口名）+ 宽高/格式/
// usage 打包成标准 descriptor 结构。
}
BufferDescriptor descriptor;
status_t error = mMapper.createDescriptor(static_cast<void*>(&descriptorInfo),
static_cast<void*>(&descriptor));
```

```
// 讲解注释⑤：descriptor 必须经 mapper 创建——同一 HAL 家族校验
// 合法性（对齐/格式组合/usage 组合），分配器只认 mapper 背书
// 的描述符。这是"谁定义类型谁校验"的职责切分。
if (error != NO_ERROR) return error;
if (mAidlAllocator) { // 讲解注释⑥：AIDL 优先
AllocationResult result;
auto status = mAidlAllocator->allocate(descriptor, bufferCount, &result);
// vendor HAL 在这里真正分配 dma-buf（可能来自池），返回
// n 个 native_handle + 公共 stride。
if (!status.isOk()) {
error = status.getExceptionCode();
if (error == EX_SERVICE_SPECIFIC) error = status.getServiceSpecificError();
if (error == OK) error = UNKNOWN_ERROR;
} else {
if (importBuffers) {
for (uint32_t i = 0; i < bufferCount; i++) {
auto handle = makeFromAidl(result.buffers[i]); // 讲解注释⑦
error = mMapper.importBuffer(handle, &outBufferHandles[i]);
// AIDL 版 handle 是 parcel 化的 fd 数组：makeFromAidl
// 还原 native_handle，importBuffer 在本进程 dup fd 并
// 建立进程内映射元数据——"跨进程跳 2"就在这行。
native_handle_delete(handle);
if (error != NO_ERROR) { /* 回滚已导入的 i 个 */ break; }
}
} else { /* importBuffers=false：只回传裸 handle */ }
}
}
...
}
```

设计洞察 · 逐段精读：为什么 allocator 走 binder 而 mapper 走 passthrough

分配是低频重操作（一次几十微秒到毫秒级），进程边界换来的隔离（vendor 私有堆管理/内存记账）完全值得；映射与 lock 是每帧热路径且 CPU 侧要直接摸到虚拟地址，passthrough 让 mapper 的 so 在调用者进程内执行——跨进程只剩内核 dma-buf 语义本身。这也解释了 fd 的角色：**跨设备的 fence 用 fd、跨进程的内存也用 fd**，binder 只递 fd 不搬数据（书3 §六的图形特例）。

11.2 dequeue 内的分配现场与失败回滚

cpp libs/gui/BufferQueueProducer.cpp:547-576（节选+讲解注释）

```
if (returnFlags & BUFFER_NEEDS_REALLOCATION) { // 讲解注释①：找槽循环
// 判定该槽需重新分配（尺寸/格式/usage 变了或全新槽）才走这里
BQ_LOGV("dequeueBuffer: allocating a new buffer for slot %d", *outSlot);
sp<GraphicBuffer> graphicBuffer = new GraphicBuffer(
width, height, format, BQ_LAYER_COUNT, usage,
{mConsumerName.string(), mConsumerName.size()}); // 讲解注释②
// 构造即分配：BQ_LAYER_COUNT=1；未参是 requestorName（dumpsys
// 里看到的槽位主人名，排障利器）。内部走到 11.1 的全链。
status_t error = graphicBuffer->initCheck();
{ // Autolock scope——重新拿 mCore->mMutex（分配在锁外！见③）
std::lock_guard<std::mutex> lock(mCore->mMutex);
if (error == NO_ERROR && !mCore->mIsAbandoned) {
graphicBuffer->setGenerationNumber(mCore->mGenerationNumber);
```

```
mSlots[*outSlot].mGraphicBuffer = graphicBuffer; // 讲解注释④
}
mCore->mIsAllocating = false; // 讲解注释⑤：分配串行化栅栏
mCore->mIsAllocatingCondition.notify_all(); // 撤除——3.1.1 里
// waitWhileAllocatingLocked 的等待者们在此被集体唤醒。
if (error != NO_ERROR) {
mCore->mFreeSlots.insert(*outSlot); // 讲解注释⑥：分配失败
mCore->clearBufferSlotLocked(*outSlot); // 槽位退回"从未
BQ_LOGE("dequeueBuffer: createGraphicBuffer failed"); // 分配"态
return error; // ——dequeue 报错给 App，本章链路显形。
}
}
...
}
```

三处设计洞察：①**分配在锁外**——new GraphicBuffer 不持 mCore->mMutex（毫秒级操作锁住会卡死 acquire/release 全家），靠 mIsAllocating 标志 + 条件变量实现"其他 dequeue 等分配完成而非等锁"；②**世代号**——setGenerationNumber 防止跨配置复用旧 buffer（分辨率切换后旧槽直接作废）；③**失败即回滚**——槽位退回 mFreeSlots 而不是悬空，账本不变式（第三章 3.3）在任何路径都不破坏。

Gralloc 世代	接口形态	分配入口	本册要点
2.0	HIDL	IAllocator 2.0	android 8 引入 HIDL 化
3.0	HIDL	IAllocator 3.0	android 10，裸句柄语义收紧
4.0	HIDL	IAllocator 4.0	android 11，multi-plane YUV/CPU 访问元数据
AIDL（13 主路径）	AIDL	IAllocator.allocate(descriptor, count)	mainline 可更新；fd 经 binder 直传

设计洞察 · 设计洞察：分配失败是常态而非异常

11.1 的 AllocationError 异常、11.2 的槽位回滚、3.1.1 的等待唤醒构成一条完整失败语义链：HAL 拒绝（碎片/超预算）→ BQ 账本回滚 → App 收到错误码。调优视角：dumpsys SurfaceFlinger 的缓冲清单与 lsof 的 dmabuf 条目对不上号时，先查 requestorName 对应进程是否有分配失败日志（BQ_LOGE 前缀）——大分辨率+高 usage 组合最易触发。

交叉阅读：第二章（usage bits 与懒分配心法）、第三章（找槽循环与 mIsAllocating 等待语义）、第八章（lockAsync/unlockAsync 与 fence 的配对——分配只是把内存立起来，时序安全全靠 fence）。

第十二章 RenderEngine：CLIENT 层合成的 GPU 管线

设计洞察 · 学习目标

1. 核实并陈述 Android 13 RenderEngine 默认后端（SKIA_GL_THREADED）与 sysprop 回退机制；2. 画出 CompositionEngine::present→Output::present→composeSurfaces→drawLayers 的四段调用链；3. 解释"缓存资源而非结果"的两个实现（AutoBackendTexture 与 ClientCompositionRequestCache）；4. 定位 CLIENT 合成慢时的 systrace 证据与调优旋钮

第七章 7.2 给了 RenderEngine 的鸟瞰，本章核实默认后端并下探到绘制现场。**核实结论：Android 13 主线默认** `SKIA_GL_THREADED`——`RenderEngine::Builder` 的成员默认值就是它（libs/renderengine/include/renderengine/RenderEngine.h:326-327），即 **Skia 后端 + 独立渲染线程**；GLES 传统后端未被删除，可用属性 `debug.renderengine.backend`（RenderEngine.h:38）回退：`gles/threaded/skiagl/skiaglthreaded` 四值在 SurfaceFlinger.cpp:742-758 逐一比对后覆盖 Builder。工厂分发在 libs/renderengine/RenderEngine.cpp:28-52：SKIA_GL 直接构造 `SkiaGLRenderEngine`，SKIA_GL_THREADED 再包一层 `RenderEngineThreaded`（任务队列 + 专用 GL 上下文）。

链路位置：第六章 composite 的注释②把帧交给 `mCompositionEngine->present(refreshArgs)`（SF/SurfaceFlinger.cpp:2226 调用，实现在 CompositionEngine/src/CompositionEngine.cpp:91）→ 全输出 prepare → `output->present`（CompositionEngine/src/Output.cpp:429）→ HWC validate 判定 CLIENT 层后 `finishFrame` → `composeSurfaces`（:1150）→ `renderengine->drawLayers(...)` → Skia/GL 落地。**SF 前端（Layer 世界观）与 CE 后端（合成执行）在此分界**——这正是第七章"职责分离的合成器版"的实体。

12.1 Output::present 八步精读（真实源码）

cpp SF/CompositionEngine/src/Output.cpp:429-451（节选+讲解注释）

```
void Output::present(const compositionengine::CompositionRefreshArgs& refreshArgs) {
ATRACE_CALL();
updateColorProfile(refreshArgs); // 讲解注释①：本帧输出色彩档
//  （广色域内容进来时选 sRGB/P3/HDR 档——第 7.2 色彩管理的入口）
updateCompositionState(refreshArgs); // 讲解注释②：把 refreshArgs
// 里的 LayerFE 前端状态灌进各 OutputLayer 的合成状态
planComposition(); // 讲解注释③：合成策略规划——
// Planner 对每层打 "capability"（无翻转/翻转/混合复杂度），
// 与 HWC validate 的输入在此汇合（第七章三步语义的②）
writeCompositionState(refreshArgs); // 讲解注释④：把各层状态写
// 给 HWC 层句柄（层序/几何/dataspace/alpha 的 HWC 命令流）
setColorTransform(refreshArgs);
beginFrame(); // 讲解注释⑤：DisplaySurface 的
// buffer 环（onFrame/前一个 present fence 等待）从这里开始
GpuCompositionResult result;
const bool predictCompositionStrategy = canPredictCompositionStrategy(refreshArgs);
if (predictCompositionStrategy) {
result = prepareFrameAsync(refreshArgs); // 讲解注释⑥：策略预测
} else { // 路径（缓存上帧策略
prepareFrame(); // 讲解注释⑦：HWC validate/create/accept 在此
} // ——DEVICE/CLIENT 清单定型（第七章决策的裁决点）
devOptRepaintFlash(refreshArgs);
```

```
finishFrame(refreshArgs, std::move(result)); // 讲解注释⑧：CLIENT
// 层在此经 composeSurfaces→drawLayers 画进 clientTarget
postFramebuffer(); // HWC presentDisplay：送显 + present fence
renderCachedSets(refreshArgs);
}
```

12.2 composeSurfaces：从层清单到 drawLayers（真实源码）

cpp SF/CompositionEngine/src/Output.cpp:1150-1236（节选+讲解注释）

```
std::optional<base::unique_fd> Output::composeSurfaces(const Region& debugRegion,
const compositionengine::CompositionRefreshArgs& refreshArgs,
std::shared_ptr<renderengine::ExternalTexture> tex, base::unique_fd& fd) {
const auto& outputState = getState();
const TracedOrdinal<bool> hasClientComposition = {"hasClientComposition",
outputState.usesClientComposition};
if (!hasClientComposition) { // 讲解注释①：全 DEVICE 直通帧直接
setExpensiveRenderingExpected(false); return base::unique_fd(); // 空转
}
// 讲解注释②：装配 DisplaySettings——物理裁剪/旋转/输出
// dataspace/亮度（nits）与 renderIntent：HDR tone-map 的
// 全部输入在这几行定齐（CLIENT 路径画质差异的根源）。
renderengine::DisplaySettings clientCompositionDisplay;
clientCompositionDisplay.physicalDisplay = outputState.framebufferSpace.getContent();
clientCompositionDisplay.clip = outputState.layerStackSpace.getContent();
clientCompositionDisplay.outputDataspace = mDisplayColorProfile->hasWideColorGamut()
? outputState.dataspace : ui::Dataspace::UNKNOWN;
clientCompositionDisplay.maxLuminance =
mDisplayColorProfile->getHdrCapabilities().getDesiredMaxLuminance();
...
// 讲解注释③：生成逐层 LayerSettings（几何/alpha/模糊半径/阴影/
// 纹理+fence）——"合成计划"的 GPU 版清单。
std::vector<LayerFE::LayerSettings> clientCompositionLayers =
generateClientCompositionRequests(supportsProtectedContent,
clientCompositionDisplay.outputDataspace,
clientCompositionLayersFE);
// 讲解注释④：请求缓存——本帧请求指纹与上帧相同（静态 UI 垫在
// 视频上的常见形态）则直接复用上帧 clientTarget，GPU 全帧休眠：
if (mClientCompositionRequestCache) {
if (mClientCompositionRequestCache->exists(tex->getBuffer()->getId(),
clientCompositionDisplay,
clientCompositionLayers)) {
ATRACE_NAME("ClientCompositionCacheHit");
outputCompositionState.reusedClientComposition = true; // 讲解注释⑤
setExpensiveRenderingExpected(false);
return base::unique_fd(); // 命中：连 drawLayers 都不调
}
ATRACE_NAME("ClientCompositionCacheMiss");
mClientCompositionRequestCache->add(...);
}
// 讲解注释⑥：GPU 提频——模糊/色彩变换是重 shader，合成窗口内
// 临时拉频，drawLayers 完成后复位（perfetto 里 GPU freq 尖峰
// 与 hasClientComposition 轨对齐的原因）。
...
base::unique_fd drawFence = renderengine.drawLayers(...); // 讲解注释⑦
```

```
// 异步提交：返回 fence 而非等待——第八章"事后声明"的又一实例。
}
```

12.3 SkiaGLRenderEngine：绘制与提交

cpp libs/renderengine/skia/SkiaGLRenderEngine.cpp:780-810 与 1240-1252（合并节选+讲解注释）

```
// drawLayers 内
auto grContext = getActiveGrContext(); // 讲解注释①：GrContext 即
// Skia 的 GPU 上下文（GL 后端）；THREADED 模式下它活在渲染线程。
std::shared_ptr<AutoBackendTexture::LocalRef> surfaceTextureRef;
// 讲解注释②：clientTarget（输出缓冲）包成 AutoBackendTexture——
// 缓存 keyed on 缓冲 id：同一 GraphicBuffer 每帧复用同一 GL 纹理，
// 免反复 create/destroy（缓存资源而非结果的第一现场）。
surfaceTextureRef = std::make_shared<AutoBackendTexture::LocalRef>(grContext, ...);
sk_sp<SkSurface> dstSurface = surfaceTextureRef->getOrCreateSurface(dstDataspace, grContext);
SkCanvas* dstCanvas = dstSurface->getCanvas();
initCanvas(dstCanvas, display); // 讲解注释③：裁剪/旋转/色彩矩阵
// 一次性铺底（SkiaGLRenderEngine.cpp:666）。
for (const auto& layer : layers) { // 讲解注释④：Z 序逐层
if (layer.backgroundBlurRadius > 0 || layer.blurRegions.size() > 0) {
auto blurredImage = mBlurFilter->generate(grContext, ...); // :955
// 背景模糊是独立 pass（降采样→模糊→上采样）——BIG 最贵
// 的 CLIENT 特性，直通必败项（第七章决策变量）。
}
if (layer.shadow.length > 0) { drawShadow(canvas, rrect, layer.shadow); } // :1002
std::shared_ptr<AutoBackendTexture::LocalRef> imageTextureRef = nullptr;
// 讲解注释⑤：层内容纹理同样走 AutoBackendTexture 缓存；
sk_sp<SkImage> image = imageTextureRef->makeImage(layerDataspace, alphaType, grContext);
dstCanvas->drawImage(image, 0, 0, SkSamplingOptions(), &paint); // 讲解注释⑥
// 混合模式/圆角裁剪/变换全部表达为 Skia paint —— 一条
// GPU 侧 op 流，无逐层 GL 状态机切换的旧 GLES 开销。
}
// ...
const bool success = grContext->submit(requireSync); // :1249
// 讲解注释⑦：requireSync=false 时异步提交、fence 稍后回填——
// SF 主循环不陪 GPU；sync 只在需要立即读回的调试路径。
```

RenderEngineType	数值	后端	线程模型	13 状态
GLES	1	自有 GL 管线	SF 主循环内	遗留（sysprop 可回退）
THREADED	2	GLES	独立渲染线程	遗留
SKIA_GL	3	Skia GL	SF 主循环内	过渡形态
SKIA_GL_THREADED	4	Skia GL	独立渲染线程	**主线默认**（RenderEngine.h:327）

设计洞察 · 设计洞察：Skia 化买到了什么

三笔账：①Shader 统一——模糊/圆角/色彩管理从手写 GLSL 迁到 Skia 效果图，HDR tone-map 与 sRGB 合流；②线程隔离——RenderEngineThreaded 让主循环只做"排任务收 fence"，GPU 长任务不再拉住 SF 主循环（对应第十章 SF 自主节拍的余量设计）；③可移植——SkiaVK 后端同构可插。代价是缓存复杂度上移（AutoBackendTexture 生命周期跨帧），泄漏观测点：`dumpsys SurfaceFlinger` 的纹理条目与 GL 内存水位。

交叉阅读：第七章 7.2（鸟瞰与色彩管理）、第七章三步语义（validate 在 12.1 注释⑦定型）、第八章（drawLayers 的 fence 语义）、第六章 composite（refreshArgs 的装配现场在 SurfaceFlinger.cpp:2179-2225）。

第十三章 帧的生命周期闭环：一块 buffer 的六站之旅

设计洞察 · 学习目标

1. 按 dequeue→queue→提交→latch→present→release 六站复述一块 buffer 的完整旅程并标注每站的执行者与状态迁移；2. 追踪 release fence 从 postComposition 回流到 App dequeue 的完整回调链（四段跳）；3. 解释 latch 时 acquire fence 未到、present 后才 release 的两处"迟到"设计；4. 用 perfetto FrameTimeline 的三段回执对号一次真实掉帧

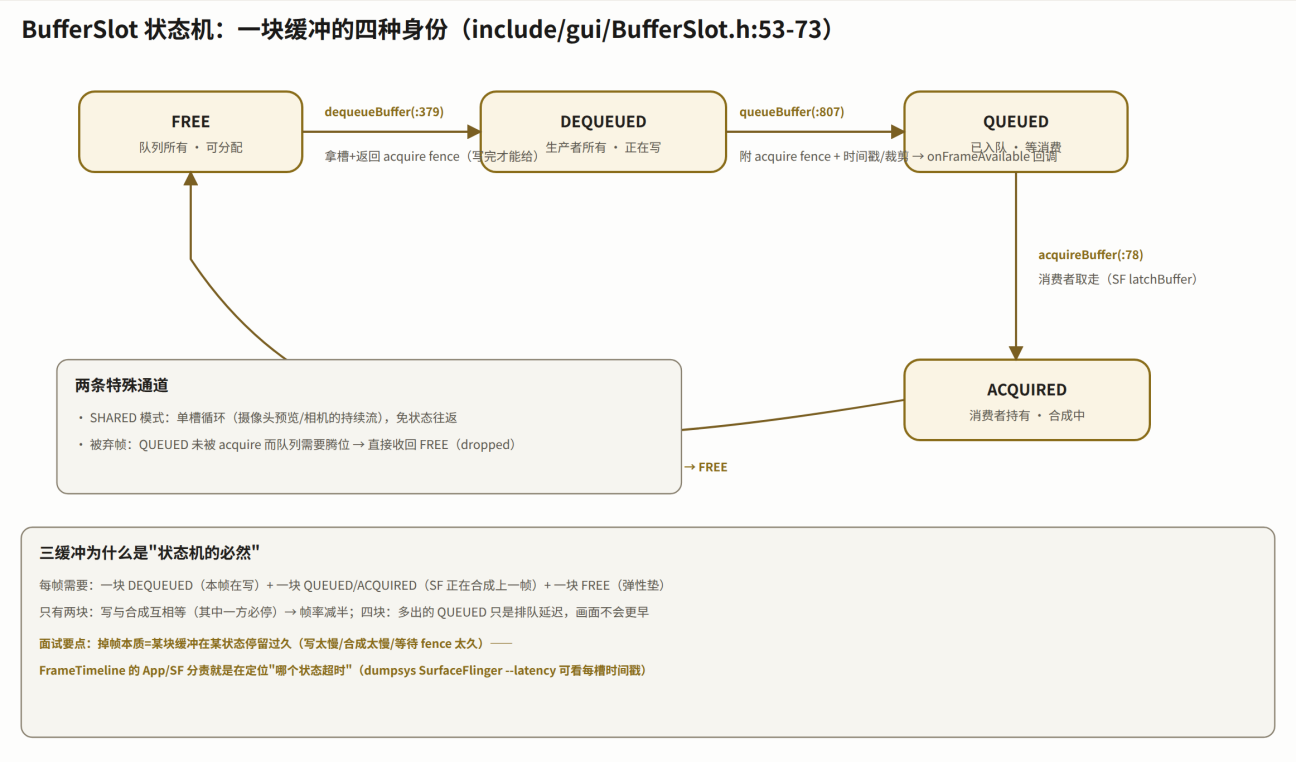


图 7 BufferSlot 状态机：FREE/DEQUEUED/QUEUED/ACQUIRED 四态流转与 SHARED 特殊通道，三缓冲的必然性

前三、四、六章分别看过状态机、提交与 SF 循环的局部，本章把六站串成一条时间线——**一块 buffer 从被 App 借走到还回 FREE 的全旅程**。六站与状态迁移：

站	执行者	状态迁移	携带的 fence	回执落点
一 dequeue	App + BQ Producer	FREE→DEQUEUED	交出 release fence（上一任读者承诺）	—
二 queue	App + BQ Producer	DEQUEUED→QUEUED	登记 acquire fence（写完承诺）	FrameEventHistory
三 提交	BLAST（BBQ:635）	QUEUED（影子队列）	随 setBuffer 进事务	apply 即返
四 latch	SF commit/latchBuffers	QUEUED→ACQUIRED	acquire fence 未到则用旧帧	latchTime 回执
五 present	composite→postComposition	ACQUIRED（在屏）	present fence（上屏事实）	presented 回执
六 release	postComposition→BBQ	ACQUIRED→FREE	release fence 回流 App	released 回执

13.1 第四站现场：latchBuffers 全文（真实源码）

cpp SF/SurfaceFlinger.cpp:3532-3607（节选+讲解注释）

```
bool SurfaceFlinger::latchBuffers() {
ATRACE_CALL();
const nsecs_t latchTime = systemTime(); // 讲解注释①：latchTime 是
// 三段回执的中段——BLAST TransactionCompleteCallback 的原料
bool visibleRegions = false;
bool frameQueued = false;
bool newDataLatched = false;
const nsecs_t expectedPresentTime = mExpectedPresentTime.load();
// 讲解注释②：先遍历整棵 Layer 树做 doTransaction（几何结算）；
// hasReadyFrame 才有资格进合成集合；shouldPresentNow 不满足
//  （desiredPresentTime 在未来）则用空 damage——帧预约语义。
mDrawingState.traverse([&](Layer* layer) {
if (layer->clearTransactionFlags(eTransactionNeeded) || mForceTransactionDisplayChange) {
const uint32_t flags = layer->doTransaction(0);
if (flags & Layer::eVisibleRegion) mVisibleRegionsDirty = true;
}
if (layer->hasReadyFrame()) {
frameQueued = true;
if (layer->shouldPresentNow(expectedPresentTime)) {
mLayersWithQueuedFrames.emplace(layer);
} else {
ATRACE_NAME("!layer->shouldPresentNow()");
layer->useEmptyDamage();
}
} else { layer->useEmptyDamage(); }
});
// 讲解注释③：离屏层（虚拟屏/录屏）独立 latch+release——否则
// 其生产者的 dequeueBuffer 会因无人还槽而永久阻塞（3.1.1 的
// 背压在离屏世界同样生效，注释原文直说 "dequeueBuffer doesn't
// block indefinitely"）。
```

```
for (Layer* offscreenLayer : mOffscreenLayers) {
offscreenLayer->traverse(LayerVector::StateSet::Drawing,
[&](Layer* l) { l->latchAndReleaseBuffer(); });
}
if (!mLayersWithQueuedFrames.empty()) {
// 讲解注释④：逐层 latchBuffer（BufferStateLayer::updateTexImage
// 在此被调，见 13.2）——acquire fence 未到不阻塞，直接
// latch "已 QUEUED 的最新状态"；成功的进 pendingRefresh。
Mutex::Autolock lock(mStateLock);
for (const auto& layer : mLayersWithQueuedFrames) {
if (layer->latchBuffer(visibleRegions, latchTime, expectedPresentTime)) {
mLayersPendingRefresh.push_back(layer);
newDataLatched = true;
}
layer->useSurfaceDamage();
}
}
mVisibleRegionsDirty |= visibleRegions;
// 讲解注释⑤：有帧在途但本帧没 latch 成（预约未来帧/被拒）→
// 下个 vsync 再醒一次查岗——scheduleCommit 的节拍续约。
if (frameQueued && (mLayersWithQueuedFrames.empty() || !newDataLatched)) {
scheduleCommit(FrameHint::kNone);
}
...
return !mLayersWithQueuedFrames.empty() && newDataLatched; // 讲解注释⑥：
// false→commit 返回 false→composite 不执行（第十章 10.3 注释④）
}
```

13.2 latch 内核：BufferStateLayer 的两步记账（真实源码）

cpp SF/BufferStateLayer.cpp:635-703（节选+讲解注释）

```
status_t BufferStateLayer::updateTexImage(bool&, nsecs_t latchTime, nsecs_t) {
const State& s(getDrawingState()); // 讲解注释①：BLAST 模型下没有
// BufferQueue 消费回调——buffer 直接躺在事务的 DrawingState 里，
// "acquire" 已在事务应用时完成（对比 BufferQueueLayer 的
// updateTexImage→BufferLayerConsumer::updateTexImage 老路）。
if (!s.buffer) { /* 只有 bgColorLayer 的空帧：仅登记 latchTime */ }
for (auto& handle : mDrawingState.callbackHandles) {
if (handle->frameNumber == mDrawingState.frameNumber)
handle->latchTime = latchTime; // 讲解注释②：回执①落账
}
const uint64_t bufferId = mDrawingState.buffer->getId();
const auto acquireFence = std::make_shared<FenceTime>(mDrawingState.acquireFence);
mFlinger->mTimeStats->setAcquireFence(layerId, frameNumber, acquireFence);
mFlinger->mTimeStats->setLatchTime(layerId, frameNumber, latchTime); // 讲解注释③
mFlinger->mFrameTracer->traceFence(layerId, bufferId, frameNumber, acquireFence,
FrameTracer::FrameEvent::ACQUIRE_FENCE);
mFlinger->mFrameTracer->traceTimestamp(layerId, bufferId, frameNumber, latchTime,
FrameTracer::FrameEvent::LATCH); // 讲解注释④
// FrameTracer/TimeStats 双通道——perfetto 的 fence 轨与
// dumpsys --timestats 的数据源都在这里上报。
std::deque<sp<CallbackHandle>> remainingHandles;
mFlinger->getTransactionCallbackInvoker()
.addOnCommitCallbackHandles(mDrawingState.callbackHandles, remainingHandles);
```

```
// 讲解注释⑤：onCommit 段回执先走一批（第五章"三重语义"的
// 可回执性在 SF 侧的分段实现）。
return NO_ERROR;
}
status_t BufferStateLayer::updateActiveBuffer() { // :688
const State& s(getDrawingState());
if (s.buffer == nullptr) return BAD_VALUE;
if (!mBufferInfo.mBuffer || !s.buffer->hasSameBuffer(*mBufferInfo.mBuffer))
decrementPendingBufferCount(); // 讲解注释⑥：App 侧在途计数减一
mPreviousReleaseCallbackId = {getCurrentBufferId(), mBufferInfo.mFrameNumber};
mBufferInfo.mBuffer = s.buffer; // 讲解注释⑦：active 槽换血：
mBufferInfo.mFence = s.acquireFence; // 旧 bufferId+帧号存为
mBufferInfo.mFrameNumber = s.frameNumber; // previous——第六站
return NO_ERROR; // release 就靠这组 id 找到旧帧
}
```

设计洞察 · 逐段精读：两处"迟到"是同一设计

latch 时 acquire fence 未到→照用（第六章"为什么 latch 不等 fence"）；present 后才 release→旧 buffer 在屏期间绝不还人。共同点：**所有权与就绪解耦**——状态机管所有权（ACQUIRED 归 SF），fence 管就绪（内核信号量）。于是任何一环慢都不传染：GPU 慢→latch 用旧帧；面板慢→present fence 晚响但状态机不回退。诊断时把两者分开问：jank 源于"等谁"，先看 fence 信号时间与状态机时间戳的差值。

13.3 第六站：release fence 的四段回流（真实源码）

cpp SF/BufferStateLayer.cpp:142-175 与 GUI/BLASTBufferQueue.cpp:390-443（合并节选+讲解注释）

```
void BufferStateLayer::releasePendingBuffer(nsecs_t dequeueReadyTime) { // 第五站后
for (const auto& handle : mDrawingState.callbackHandles) {
handle->transformHint = mTransformHint; // 讲解注释①：把本帧的
handle->dequeueReadyTime = dequeueReadyTime; // 旋转提示/还槽
handle->currentMaxAcquiredBufferCount = // 时刻/新在途上限随
mFlinger->getMaxAcquiredBufferCountForCurrentRefreshRate(mOwnerUid);
} // 回执带给 App——dequeue 侧据此调整阻塞预算（帧率切换时自动改）
...
std::vector<JankData> jankData; // 讲解注释②：积压的 jank 分类随
jankData.reserve(mPendingJankClassifications.size()); // 回执一起
while (!mPendingJankClassifications.empty() && // 发还 App——
mPendingJankClassifications.front()->getJankType().value()) {
... // FrameTimeline 的责任判定最终"寄"回生产者
}
mFlinger->getTransactionCallbackInvoker().addCallbackHandles(
mDrawingState.callbackHandles, jankData); // 讲解注释③：第二段跳
sp<Fence> releaseFence = Fence::NO_FENCE;
for (auto& handle : mDrawingState.callbackHandles) {
if (handle->releasePreviousBuffer && mDrawingState.releaseBufferEndpoint == handle->listener
) {
releaseFence = handle->previousReleaseFence ? handle->previousReleaseFence
: Fence::NO_FENCE; // 讲解注释④
break; // release fence 本体（SF 侧上帧读完的承诺）挂上
}
}
}
// ——TransactionCallbackInvoker::sendCallbacks（:185）经
```

```
// BackgroundExecutor（:236）投递到 App 进程——
void BLASTBufferQueue::releaseBufferCallback(const ReleaseCallbackId& id, ...) { // 第三段跳
...
releaseBuffer(id, releaseFence); // 讲解注释⑤：找到影子队列里该
// callbackId 对应的 buffer（第四段跳）：
}
void BLASTBufferQueue::releaseBuffer(const ReleaseCallbackId& callbackId, const sp<Fence>&
releaseFe
nce) {
auto it = mSubmittedBuffers.find(callbackId);
if (it == mSubmittedBuffers.end()) { BQA_LOGE("ERROR: ..."); return; }
mSubmittedBuffers.erase(it); // 讲解注释⑥：影子队列销账
mBufferItemConsumer->releaseBuffer(it->second, releaseFence); // 讲解注释⑦
// BQA::releaseBuffer→BufferQueueProducer::releaseBuffer：
// 槽位 ACQUIRED→FREE，dequeue 的 waitForFreeSlotThenRelock
// 在此被唤醒——状态机闭环，第一站解除背压。
}
```

设计洞察 · 设计洞察：回执链为什么绕四跳

SF→TransactionCallbackInvoker→BackgroundExecutor→BBQ→BQA 看似绕远，实为三层解耦：Invoker 是"待确认事务"的登记处（present fence 齐了才发，避免半成品回执）；BackgroundExecutor 把回调执行挪出 SF 主线程（回调里可能再 apply 新事务，主线程重入即死锁）；BBQ 的 mSubmittedBuffers 是 callbackId→GraphicBuffer 的映射表（BLAST 下 App 侧不再持有 BQ 槽位概念，靠 id 对账）。**任何一跳断了，现象都是同一个：App dequeue 永久阻塞、在途计数只增不减**——perfetto 上找 TransactionCallback 轨、logcat 过 BBQ 标签即可定位断在哪层。

交叉阅读：第三章（状态机与背压等待点）、第四章（影子队列与三段回执的申请方）、第六章（commit/composite 骨架——本章给出 latch 与 postComposition 的实体）、第八章（三 fence 落点与本站时间轴对照）、第十章（唤醒本循环的节拍从哪来）。

教科书习题集（分级）

第一级：简答题（检验概念理解）

1. **BufferSlot 四态分别由谁"拥有"？相邻两态迁移时各携带哪种 fence？**

参考答案要点：FREE（BQ 账本）/DEQUEUED（生产者）/QUEUED（账本排队）/ACQUIRED（消费者）；dequeue 交出上帧 release fence，queue 登记 acquire fence，release 归还时挂新 release fence（第三章）。

2. **dequeueBuffer 可能"卡住"的两种等待分别是什么？各自等在什么机制上？**

参考答案要点：等分配（mIsAllocating + mIsAllocatingCondition，一次性）与等空闲槽（waitForFreeSlotThenRelock 的 mDequeueCondition，常态背压）——前者等别的线程分完，后者等消费者 release（第三章 3.1.1）。

3. **BLAST 的影子队列是什么？连续积压两帧时 BBQ 做什么？**

参考答案要点：App 进程内自建的小队列，帧先入队再按节拍组 Transaction 提交，提交速率与生产速率解耦；mNumFrameAvailable≥2 且在等回调时 acquireAndReleaseBuffer 主动丢旧帧——宁可跳帧不排队延迟（第四章）。

4. **三段回执（latched/presented/released）分别由谁在什么时机触发?**

参考答案要点：latchTime 在 SF latchBuffers/BufferStateLayer::updateTexImage 落账；presented 由 postComposition 的 present fence 触发（addPresentFence+sendCallbacks）；released 经 releasePendingBuffer→TransactionCallbackInvoker→BBQ releaseBufferCallback 回到 App（第十三/四章）。

5. **Android 13 主线 RenderEngine 的默认后端是什么？如何回退到传统 GLES?**

参考答案要点：默认 SKIA_GL_THREADED（Builder 默认值，RenderEngine.h:327）；`setprop debug.renderengine.backend gles` 后重启 SF（SurfaceFlinger.cpp:742-758 四值比对）（第十二章）。

6. **Android 13 的 Gralloc 中 allocator 与 mapper 各用什么接口形态？为什么分工不同?**

参考答案要点：allocator=AIDL（hardware/interfaces/graphics/allocator/aidl，HIDL 4.0 兜底），低频重操作走 binder 换隔离；mapper=HIDL 4.0 强制 passthrough，每帧热路径且要本进程虚拟地址（第十一章）。

7. **EventThread 的 VSyncRequest::Single 为什么实际会唤醒两帧?**

参考答案要点：Single 命中一帧后自动降级 SingleSuppressCallback，下一帧消费但不派发再降 None——省掉请求/响应一次跨线程往返的调度开销（EventThread.h 注释原文）（第十章）。

8. **acquire/release/present 三种 fence 各自表达什么承诺？谁交给谁?**

参考答案要点：acquire=生产者"我写完了"（queue 时随 buffer 给 SF）；release=消费者"我读完了"（release 时随槽位还给生产者）；present=HWC"这帧上屏了"（presentDisplay 后 SF 收）（第八章）。

9. **为什么 SF latch 时 acquire fence 未到也要继续（用旧帧）而不等待?**

参考答案要点：latch 阻塞会卡住 SF 主循环，代价是整帧延迟；单层内容旧一帧肉眼几乎无感——帧节拍稳定性优先于单层新鲜度，fence 的"事后声明"设计正为此（第六/八章）。

10. **VirtualDisplay 的合成节拍从哪里来？投屏卡顿为什么别查主屏 vsync?**

参考答案要点：离屏层走 commitOffscreenLayers 独立结算，虚拟屏输出到自己的 BufferQueue、由投屏协议端消费；卡顿多因该消费者 release 慢导致 dequeue 背压，与主屏节拍无关（第六章 6.3/第十三章 13.1 注释③）。

第二级：分析题（检验机制理解）

11. **追踪一块 buffer 从 dequeue 到 release 的完整调用链，标出每次跨进程跳与每段的 fence。**

参考答案要点：dequeue（App 内，release fence）→queue（App 内，acquire fence）→BBQ onFrameAvailable→apply（binder：事务+fd）→SF commit/latch（事务应用）→composite→postComposition（present fence）→releasePendingBuffer→Invoker/BackgroundExecutor→BBQ releaseBuffer（binder 回程：release fence）→BQA releaseBuffer→FREE；fd 三进三出，像素零搬运（第十三/四章）。

12. **失败推演：TransactionCallbackInvoker 的回执链在某帧断裂（回调永不送达），系统将出现什么现象？如何定位断点？**

参考答案要点：App 侧 mSubmittedBuffers 只增不减→dequeue 触发背压→应用自己卡住而 SF 正常；定位：dumpsys SurfaceFlinger 看该层 PendingFrames、perfetto 找 TransactionCallback 轨缺失、logcat 过 BBQ 标签——断在 Invoker（SF 侧无 sendCallbacks）还是 App 侧（BBQ 收到但没对上账）（第十三章 13.3）。

13. **失败推演：某显示的 HWC present fence 永不 signal，背压如何逐级传导？谁最先受罚？**

参考答案要点：commit 首先检测 framePending（previousFramePending）→返回 false 放弃本帧→scheduleCommit 重试；EventThread 侧 EventThread 仍在但 SF 消费停滞；最先受罚是 SF 的合成节拍（帧率归零），App 随后因 release 停滞耗尽在途缓冲而卡 dequeue——与 EventThread "Faking VSYNC due to driver stall" 兜底形成对照（第十/十三章）。

14. **给视频层加半透明遮罩后整机功耗上升：用第七章决策三步与第十二章说明完整因果链。**

参考答案要点：遮罩引入混合→validate 判该层/整帧 CLIENT→GPU 每帧全量重画（模糊/混合 shader 更贵且触发 GPU 提频）→直通省电三理由全部失效；且 CLIENT 输出还占一层 FB 预算，视频层可能被拖下水（第七章/第十二章 12.2 注释⑥）。

15. **跨册串联：本册 fence 与书3 Binder 的交汇点在哪里？为什么 binder 传 fence 不破坏"不搬数据"原则？**

参考答案要点：交汇在 fd 传递（BINDER_TYPE_FD/dup 到目标进程）——事务里的 acquire fence、release 回执都是 fd；binder 只递句柄，等待语义留在内核 sync 框架，数据面仍是 dma-buf 映射（第八章/书3 §六）。

16. **冷启动首帧明显 jank：用第十一章全链解释耗时构成，并给出三个缓解手段。**

参考答案要点：首帧走 mFreeSlots→真分配（AIDL binder 到 vendor HAL + importBuffer 建映射 + 可能触发 ion/dmabuf 池增长）；缓解：提高 maxDequeuedBufferCount 预热、复用 requestorName 池（SurfaceView/预分配）、避免首帧超大尺寸+复杂 usage 组合（第十一章 11.2）。

17. **dumpsys 显示某层 Composition type=CLIENT，但 systrace 上该帧没有 GPU 合成活动——推演至少两个可能原因。**

参考答案要点：①ClientCompositionRequestCache 命中（reusedClientComposition=true，上帧结果直接复用，连 drawLayers 都不调）；②策略预测路径 prepareFrameAsync 走了缓存策略；③该层属于其他 output（虚拟屏）本帧未合成（第十二章 12.2 注释④⑤）。

第三级：设计题（检验架构能力）

18. **若要实现"真逐层帧率"（24fps 视频层与 120Hz UI 同屏各自节拍），需要动哪些环节？列出三个最难的问题。**

参考答案要点：需 HWC/面板支持分区刷新（LFD）或 SF 双输出；难点①合成器时序（SF 单循环按 120 走、视频层按 1/5 帧跳过 latch 即可但面板分区要硬件配合）；②撕裂与相位对齐（两节拍漂移时的呈现边界）；③回执语义（present fence 按输出还是按层，FrameTimeline 归因模型要重写）（第七章 7.1 的对照面）。

19. **设计一个 fence 泄漏检测工具：在哪些观测点埋桩？如何把泄漏归因到具体生产者/消费者？**

参考答案要点：埋桩：BQ releaseBuffer/acquireBuffer 与 BLAST releaseBufferCallback（fd 生命周期两端）+ Fence::merge/waitForever；方法：每 fd 记录（创建栈、携带方、超时未 signal 计数），周期 dumpsys 汇总"最老未闭合 fence"及其 requestorName 链；归因用 bufferId+frameNumber 关联到槽位历史（第八章/第十三章）。

20. **为低端机（GPU 弱、带宽紧）设计合成决策的兜底策略：何时强制 CLIENT、何时强制 DEVICE，给出可度量的决策函数。**

参考答案要点：可度量输入：层数、可见面积、单层混合复杂度（模糊/纹理变换）、GPU 历史耗时（TimeStats）、present fence 延迟分位；策略：大面积视频类单层强制 DEVICE、模糊/复杂阴影强制 CLIENT 并限制并发、GPU 耗时超预算连续 N 帧则下一帧降级（简化混合：预合成到离屏再直通）；函数形如 cost(GPU 路径)=Σ层代价+带宽、cost(HWC 路径)=层数×控制器上限惩罚，取小者并带迟滞（第七章/第十二章）。

术语表

术语	解释
AIDL IAllocator	Android 13 图形分配 HAL 的新接口形态：唯一方法 allocate(descriptor,count) 返回 AllocationResult；HIDL 4.0 为兜底（Gralloc4.cpp:1195）
BLAST	App 侧提交框架：每帧组一个原子 Transaction（buffer+几何+回执）交 SF，消除内容/几何双通道错配
BufferQueue	生产者-消费者缓冲队列：64 槽账本（mSlots）+ 三条队列（mFreeSlots/mFreeBuffers/mQueue），两端同进程靠锁、数据面不走 binder

术语	解释
BufferSlot	队列中的槽位：FREE/DEQUEUED/QUEUED/ACQUIRED 四态，状态即"谁拥有谁负责"
背压	生产过快时 dequeue 阻塞等 release 的节流机制——不在协议里，在状态机里（waitForFreeSlotThenRelock）
侧带流	Sideband stream：绕过 SF 合成由硬件直叠的视频流（相机低延迟录制特权通道）
DispSyncSource	EventThread 的节拍源包装：CallbackRepeater 在 VSyncDispatch 上周期续约，workDuration/readyDuration 决定唤醒相位
dma-buf	内核 dma-buf 框架：图形缓冲的物理内存载体，跨进程/跨设备以 fd 传递、各自映射
Fence	内核 sync 对象的 fd 封装：不等待的依赖表达；wait 同步、merge 合并多路依赖（ui/Fence.cpp:48/90）
Gralloc	图形内存分配体系总称：allocator HAL 出 dma-buf、mapper HAL 负责各进程映射与 lock/unlock
HWC	硬件合成器（composer HAL）：validate 点名可直通层（DEVICE）、present 送显并交 present fence
回执三段	latched/presented/released：BLAST 事务的帧级回执链，FrameTimeline 的数据原料
latch	SF commit 阶段取每层最新已提交缓冲的动作：acquire fence 未到则用旧帧（节拍优先）
LayerHistory	SF 内的帧率投票器：采样各层提交间隔→启发式归类→全局刷新率提案（投票单位是 Layer）
Overlay 直通	层不经 GPU 合成、由显示控制器逐层叠加（DEVICE 合成）：省电省带宽省 GPU
passthrough	HAL 的同进程 so 执行模式：mapper 强制如此（每帧热路径），allocator 走 binder 不如此
present fence	HWC presentDisplay 后返回的 fence：本帧真实上屏的时刻承诺
RenderEngine	SF 进程内的 GPU 画师：13 主线默认 Skia+独立线程（SKIA_GL_THREADED）
三缓冲	dequeue/queue/acquire 各占一槽的稳态配置：写/排/读/垫四角色的最小完备——上限是谈判值非写死
VSyncPredictor	VSyncPredictor：对硬件 vsync 时戳做滑动窗口回归，外推未来节拍（调度三件套的 Tracker）
影子队列	BLASTBufferQueue 在 App 侧自建的小队列：帧先入队再按节拍提交，积压两帧即丢旧帧
脏区	本帧需要重绘/重合成的区域：commit 阶段计算，CLIENT 路径据此缩小 GPU 工作量

术语	解释
直通失败	validate 判层必须 GPU 参与（混合/缩放/格式超限）：整帧或部分层回 CLIENT 路径

排障手册：图形渲染深潜问题定位

通用路线：`dumpsys SurfaceFlinger` 定位层状态（合成方式/缓冲/在途计数）→ `perfetto` 抓 FrameTimeline 与 fence 轨归因时间 → `logcat`（BufferQueueProducer/BBQ/SurfaceFlinger 标签）对账事件 → 回到对应章节读机制。以下五个场景按"症状→定位命令→根因→修复"展开。

场景一：滚动列表周期性掉帧

症状：perfetto FrameTimeline 出现 `App Deadline Missed` 或 `SurfaceFlinger GPU Deadline Missed` 标签（FrameTimeline.cpp:98-142 的真实字串）；systrace 上 doFrame 跨过 vsync 周期线；dequeueBuffer 出现长等待条。

定位命令：`dumpsys gfxinfo <pkg>`（janky frames 比例与帧耗时直方图）；`dumpsys SurfaceFlinger --latency <layerId>` 看三列时戳（desired/present 间隔）；`dumpsys SurfaceFlinger --timestats` 看合成方式占比。

根因：三类最常见——App 主线程或 GPU 超帧预算（测量布局/上传大纹理）；三缓冲耗尽后背压放大抖动；SF 侧直通失败迫使 GPU 每帧全量重画（挤占合成窗口）。

修复：按归因分治：裁层级降过绘（App）；SurfaceView.setFrameRate 对齐节拍并预分配缓冲；给视频层去掉透明遮罩恢复直通（SF 侧，参考附录 D 实验二）。

场景二：App 卡死在 dequeueBuffer（在途缓冲耗尽）

症状：主线程栈顶停在 dequeueBuffer 相关等待；logcat 无新帧日志；perfetto 上该应用的 TransactionCallback 轨中断。

定位命令：`dumpsys SurfaceFlinger` 找对应 layer 的缓冲状态与 PendingFrames 计数；`logcat -s BufferQueueProducer BBQ` 看最后一条是 queue 还是 release；`debuggerd -b <pid>` 查 BLAST 相关线程栈。

根因：release 回执链断裂（第十三章 13.3 的四段回流：Invoker 未 sendCallbacks、BackgroundExecutor 阻塞、BBQ mSubmittedBuffers 对账失败）；或消费者进程已死但队列未 abandon（mAbandoned 未置位时 dequeue 照常等待）。

修复：沿四段回流逐跳对账找出断点；确认消费端死亡路径必须走 abandon（状态机 3.3 的 mAbandoned 语义）；避免在事务回调里同步再 apply 新事务（BackgroundExecutor 重入）。

场景三：surfaceflinger 独占高 CPU，GPU 反复做同样的合成

症状：systrace 上 sf 主线程 composite 每帧都长；GPU 频率长期高位；`dumpsys SurfaceFlinger` 中大量层 Composition type=CLIENT；无 crash 但功耗异常。

定位命令：`dumpsys SurfaceFlinger` 逐层看 type 与判定原因；`dumpsys SurfaceFlinger --timestats dump` 看 clientComposition 帧占比；对照 `dumpsys SurfaceFlinger --list` 的层清单找新增/异常层。

根因：直通全面失败——常见触发：单层含模糊或大圆角阴影、缩放+旋转组合超出控制器能力、格式不被控制器接受、超出层数上限；叠加 ClientCompositionRequestCache 未命中（层属性每帧微变）导致连复用都没有。

修复：减少每层的非直通属性（动态模糊改预渲染静态图）；合并同源小层；确认调试开关未误开（mDebugDisableHWC、devOptFlashDirtyRegionsDelay）；确需 GPU 合成的内容限制在最小脏区（第十二章 12.2）。

场景四：花屏或内容残影（fence 依赖缺失）

症状：偶发花屏块或旧内容残影；perfetto fence 轨上 acquire fence 的 signal 时间晚于消费方开始读该 buffer 的起点；复现率随 GPU 负载升高。

定位命令：`dumpsys SurfaceFlinger --dump-buffer` 看各槽 fence 与时戳（状态机现场）；内核侧 `cat /sys/kernel/debug/sync/timeline`（可用的设备上）看未 signal 的 sync 点；`logcat | grep -i fence` 捕获 merge/waitForever 告警。

根因：生产者忽略 dequeue 返回的 release fence 直接写入（依赖未挂上 GPU 编队）；或跨进程传递时 fence fd 未正确 dup/提前 close（第八章口诀：花屏查 release 依赖）。

修复：核对自定义管线把 dequeue 的 fence 喂给正确的等待 API（GPU 侧 sync 语义）；跨进程只用 binder fd 通道（书3 §六）；给关键路径加 waitForever 的日志名便于归因（Fence.cpp:60）。

场景五：大图加载与首帧纹理上传慢

症状：App Deadline Missed 集中在冷启动或首次进入页面；systrace 上纹理 upload 条目长；gfxinfo 直方图出现孤立的首帧尖峰。

定位命令：`dumpsys gfxinfo <pkg> framestats` 取最近 120 帧细节；`dumpsys meminfo <pkg>` 观察图形内存涨落；分配慢时 `logcat | grep -i gralloc` 查分配失败/降级日志（第十一章 11.2 注释⑥）。

根因：大尺寸 RGBA 的上传或首次 mmap 建页 + stride 对齐损耗；冷启动叠加首帧 Gralloc 真分配（AIDL binder + importBuffer 全链）。

修复：预解码降采样并让请求尺寸对齐 stride；inBitmap 复用位图内存；预热手段（提前 dequeue 一次触发分配走 mFreeBuffers 复用道）；媒体内容走硬件解码直出 GraphicBuffer 免软件

上传（第二章/第十一章）。

10.1 帧率策略实战：如何实现 120fps 视频播放

text 帧率策略完整链（读码归纳）

```
App 层:
SurfaceView.setFrameRate(24f, FRAME_RATE_COMPATIBILITY_FIXED_SOURCE)
→ 通知 SurfaceFlinger "这层想要 24fps"
→ BLASTBufferQueue.onFrameAvailable 时携带帧间隔信息
SF 层:
LayerHistory 采样各层提交间隔（每次 composite 后记录）
→ 发现视频层稳定 24fps → 提案面板降到 24Hz?
→ 不！↑ 只改"建议值"，实际由 HWC 驱动 + 面板控制器决定
→ 大部分机型维持全局刷新率（60/90/120Hz 不切换）
→ 少数 LFD 屏（低帧率显示）支持分区帧率（仪表 60Hz + 乘客 24Hz）
面试考点:
Q 为什么不做逐层帧率?
A ①硬件 LFD 未普及 ②合成器仍需同步（帧率不一致会撕裂）
③运维复杂度高（用户不知道当前是几 Hz）
```

10.2 SurfaceView 深度解析：独立画布的代价

text SurfaceView 的三层视图（读码归纳）

```
窗口层（App 的 ViewRootImpl 管理）
└── SurfaceView（一个空的 View 占位——打洞/透明/不绘制）
└── 独立 Surface（SurfaceControl + BufferQueue → SF 合成）
（有自己的节拍——不受 ViewRootImpl 的 Choreographer 控制）
三个 trade-off:
① 独立刷新：视频帧率 ≠ UI 帧率 → 省电（GPU 不画重复帧）
② 独立缓冲：三缓冲独立于 UI 三缓冲 → 内存多 3 份 GraphicBuffer
③ 合成延迟：多一层合成交付 → 延迟比 TextureView 高 ~5ms
TextureView 的反面：内容画进 UI 的 Surface → 可动画/变换/半透明
→ 但每次新帧要 GPU 合成到 UI 图层 → CPU/GPU 开销高于 SurfaceView
面试高频：视频预览选 SurfaceView（低延迟直通），
但视频截图选 TextureView（需要 bitmap 可读）
```

附录 E：图形渲染核心机制深度速查表

机制	一句话	源码锚点	所在册
BufferSlot 四态	FREE↔DEQUEUED↔QUEUED↔ACQUIRED	BufferSlot.h:53-73	本册
dequeueBuffer	拿槽+release fence（等上家读完）	BufferQueueProducer:379	本册
queueBuffer	DEQUEUED→QUEUED+acquire fence+时戳	BufferQueueProducer:807	本册

机制	一句话	源码锚点	所在册
acquireBuffer	QUEUED→ACQUIRED（消费者取走）	BufferQueueConsumer:78	本册
BLAST	每帧一个原子事务（buffer+几何+回执）	BLASTBufferQueue:635	本册
影子队列	BBQ 自建 App 侧小队列（提交/生产解耦）	BLASTBufferQueue	本册
三段回执	latched/presented/released（FrameTimeline 原料）	BLASTBufferQueue	本册
SF commit	应用事务+latchBuffer（fence 未到用旧帧）	SF:3119	本册
SF composite	合成计划→HWC validate→GPU 兜底→present	SF:2164	本册
HWC 决策	validate 让硬件点名直通层（DEVICE vs CLIENT）	HWComposer.cpp	本册
RenderEngine	Skia GPU 合成 CLIENT 层（每帧全量重画）	CE/renderengine/skia/	本册
Fence 三主角	acquire(写完)/release(读完)/present(上屏)	ui/Fence.cpp:48	本册
Fence merge	多路依赖合为一门	Fence.cpp:90	本册
Gralloc	allocator HAL 出 dma-buf / mapper HAL 映射	hardware/interfaces/graphics	本册
LayerHistory	帧率投票（各层提交间隔采样）	SF/Scheduler/LayerHistory	本册
AIDL BQ 2.0	媒体域跨进程直连试点（13）	hardware/interfaces/graphics/bufferqueue	本册
VirtualDisplay	无面板屏（投屏/录屏）独立节拍	SF commitOffscreenLayers	本册
MMAP no-IRQ	硬件时钟直驱（音频低延迟）	AAudio	册12